

2026년 Deep Learning Hardware 설계 경진대회

2026.02.09 (Monday)



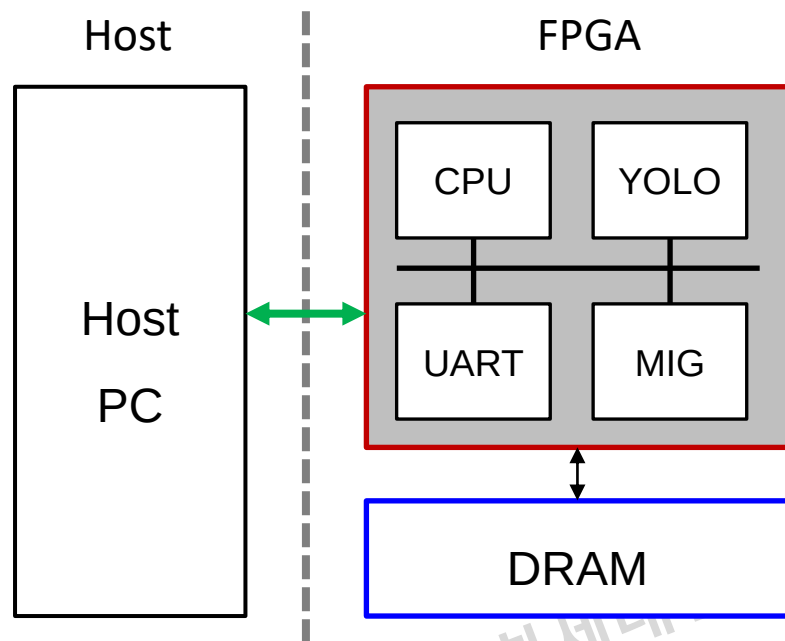
Outlines

- Recap
- Top Module
- MAC
- Buffer

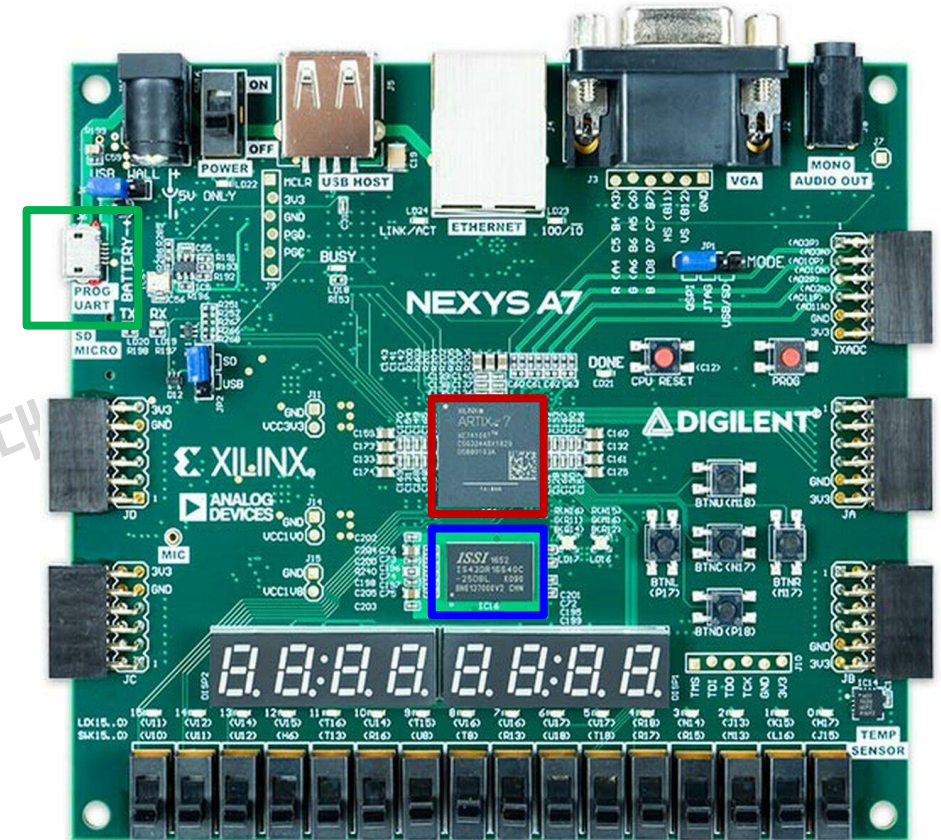
Recap:

- Last week, we show you
 - The settings for SDK
 - How we evaluate the design, including accuracy, performance, and power
 - How the system overview looks like
 - How to generate the test vectors from a quantized model and activations during an inference
 - An example design: weight buffer and test bench
- Based on the sample code and design, make your own design choice.

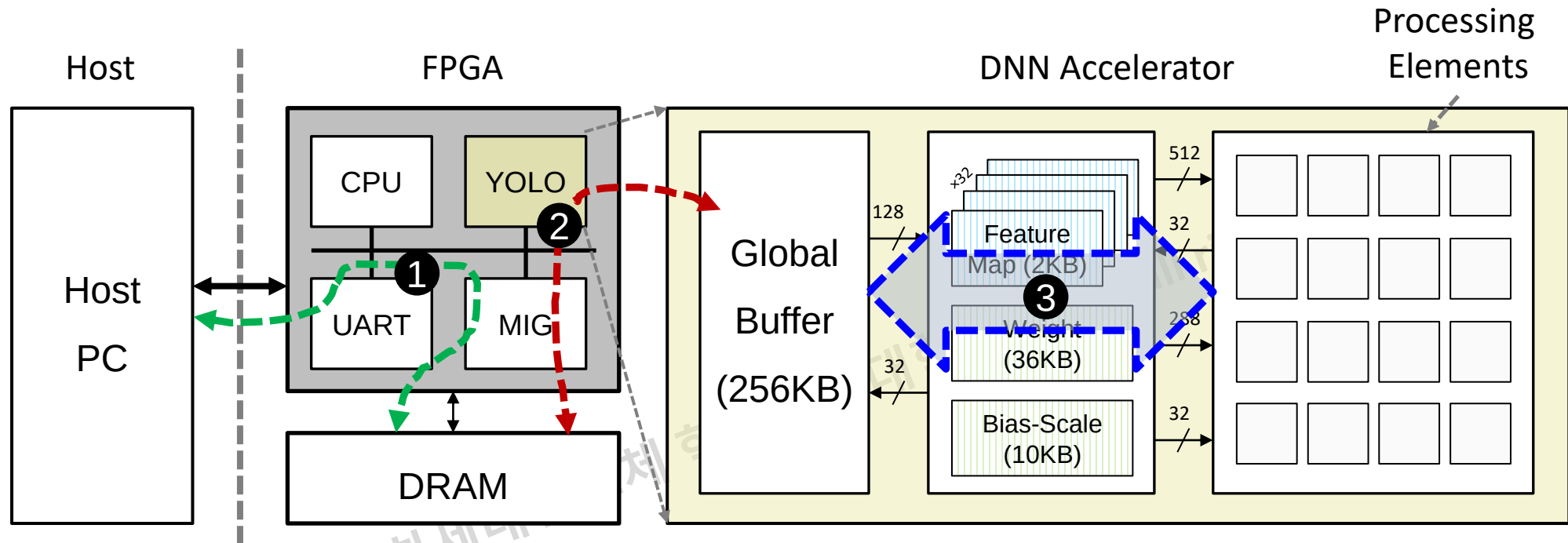
Recap: System Overview (Host-FPGA)



- The host and FPGA communicate via UART
- FPGA Side:
 - **FPGA:** CPU (MicroBlaze, UART, MIG, YOLO)
 - **DDR2 DRAM**



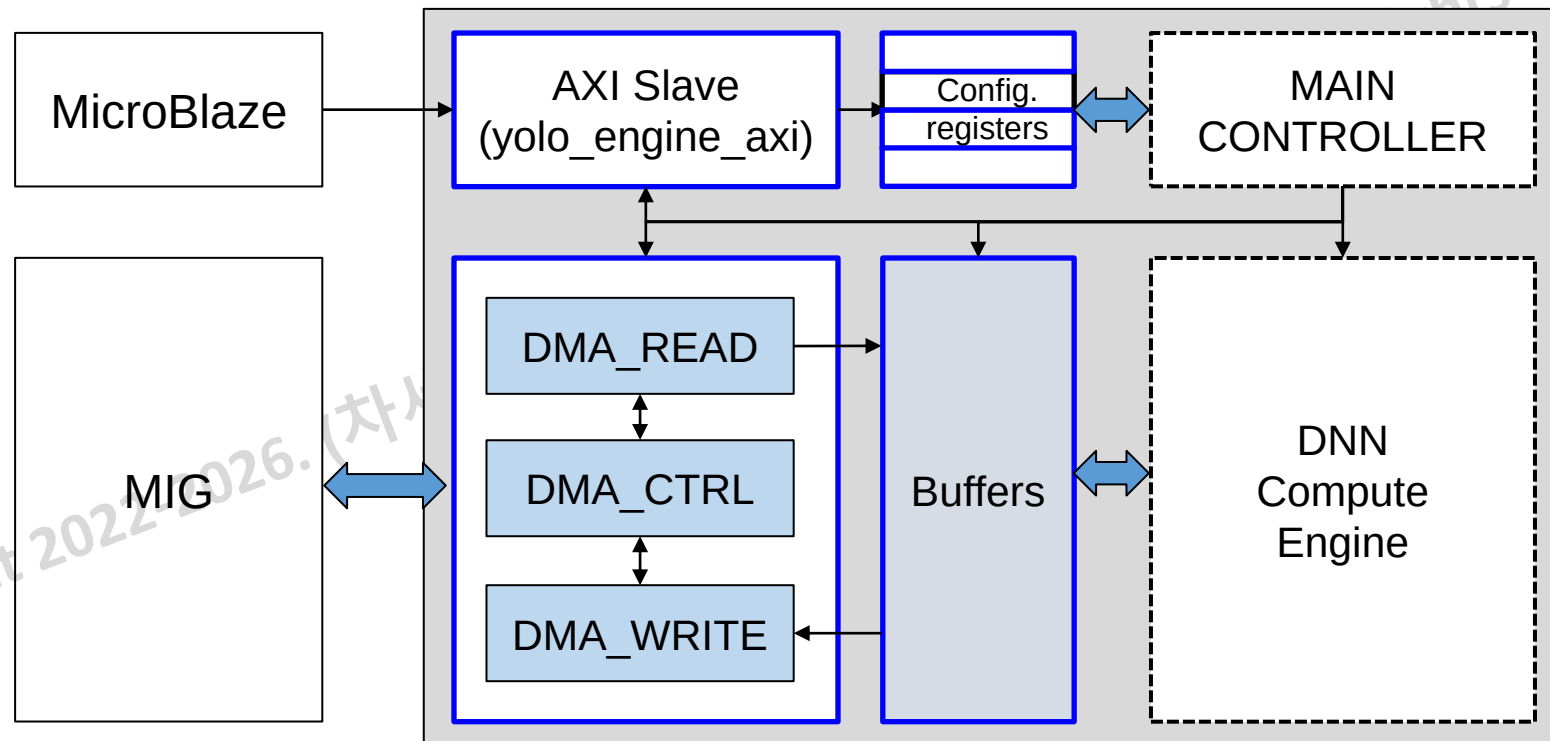
Recap: System Overview (Data Movement)



- Three main communication channels:
 - Host ↔ DRAM on FPGA via UART
 - DRAM ↔ L2 Global Buffer via DMA
 - L2 ↔ L1 Local Buffers ↔ PEs or MACs

Objective: Top Module (yolo_engine_ip)

- Top Module: yolo_engine_ip
 - AXI Slave: To communicate with the MicroBlaze CPU via control & status registers
 - AXI Master: To transfer data between DRAM and Buffers via an AXI DMA Master
 - Main controller and DNN Compute Engine



Code Structure

- Top-level

- /src Source code

- yolo_engine.v

- yolo_engine_axi.v

- axi_dma_read.v, axi_dma_write.v, axi_dma_ctrl.v

- /sim Testbench and data for simulation

- sim/yolo_engine_tb.v: A test bench for the top module

- /arxiv Screen-captured results

Simulation

- /sim/inout_data_sw/log_feamap Feature maps from SW simulation (Hex format)

- /sim/inout_data_sw/log_param Weight maps from SW simulation (Hex format)

- /sim/inout_data_hw Output for HW simulation

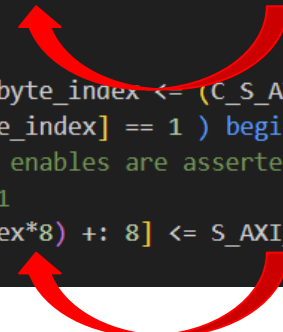
- /sim/sim_dram_model Code for a simple external memory model

AXI Slave: yolo_engine_axi

- This module is used as an interface to communicate with the CPU
 - Receive configurations from the CPU and return the status of the engine to the CPU

```
src > yolo_engine_axi.v
1  timescale 1 ns / 1 ps
2
3  module yolo_engine_axi #
4  (
5      // Users to add parameters here
6
7      // User parameters ends
8      // Do not modify the parameters beyond this line
9
10     // Width of S_AXI data bus
11     parameter integer C_S_AXI_DATA_WIDTH  = 32,
12     // Width of S_AXI address bus
13     parameter integer C_S_AXI_ADDR_WIDTH  = 4
14 )
15 (
16     // Users to add ports here
17     output wire [31:0] ctrl_reg0,
18     output wire [31:0] ctrl_reg1,
19     output wire [31:0] ctrl_reg2,
20     output wire [31:0] ctrl_reg3,
21     input  wire network_done,
22     input  wire network_done_led,
23     // User ports ends
24     // Do not modify the ports beyond this line
25 )
```

```
if (slv_reg_wren)
begin
    case ( axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] )
        2'h0:
            for ( byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index = byte_index+1 )
                if ( S_AXI_WSTRB[byte_index] == 1 ) begin
                    // Respective byte enables are asserted as per write strobes
                    // Slave register 0
                    slv_reg0[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8];
                end
        2'h1:
            for ( byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index = byte_index+1 )
                if ( S_AXI_WSTRB[byte_index] == 1 ) begin
                    // Respective byte enables are asserted as per write strobes
                    // Slave register 1
                    slv_reg1[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8];
                end
    endcase
end
```



Set configuration

- + Check if the CPU wants to write to the engine
- + Decode the address to update the target register

AXI Slave: yolo_engine_axi

- This module is used as an interface to communicate with the CPU
 - Receive configurations from the CPU and return the status of the engine to the CPU

```
src > yolo_engine_axi.v
1 | timescale 1 ns / 1 ps
2 |
3 | module yolo_engine_axi #
4 | (
5 |     // Users to add parameters here
6 |
7 |     // User parameters ends
8 |     // Do not modify the parameters beyond this line
9 |
10 |    // Width of S_AXI data bus
11 |    parameter integer C_S_AXI_DATA_WIDTH  = 32,
12 |    // Width of S_AXI address bus
13 |    parameter integer C_S_AXI_ADDR_WIDTH  = 4
14 | )
15 | (
16 |    // Users to add ports here
17 |    output wire [31:0] ctrl_reg0,
18 |    output wire [31:0] ctrl_reg1,
19 |    output wire [31:0] ctrl_reg2,
20 |    output wire [31:0] ctrl_reg3,
21 |    input  wire network_done,
22 |    input  wire network_done_led,
23 |    // User ports ends
24 |    // Do not modify the ports beyond this line
25 | )
```

```
assign slv_reg_rden = axi_arready & S_AXI_ARVALID & ~axi_rvalid;
always @(*)
begin
    // Address decoding for reading registers
    case ( axi_araddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] )
        2'h0 : reg_data_out <= {slv_reg0[31:2], network_done, slv_reg0[0]};
        2'h1 : reg_data_out <= slv_reg1;
        2'h2 : reg_data_out <= slv_reg2;
        2'h3 : reg_data_out <= slv_reg3;
        default : reg_data_out <= 0;
    endcase
end
```

+ At the engine:

Return the status: $\text{reg_data_out}[1] \leftarrow \text{network_done}$

+ At the CPU, get and decode the readout register

while(1) begin // Polling

 If $((\text{readout} \gg 1) \& 0x0000_0001)$

 break; // Execution is completed

 end if

end while

YOLO Engine Module: yolo_engine.v

- The top module
 - Control signals
 - i_ctrl_reg0~3
 - axi interfaces
 - Layer done signals
 - network_done
 - network_done_led

```
9 module yolo_engine #(
10     parameter AXI_WIDTH_AD = 32,
11     parameter AXI_WIDTH_ID = 4,
12     parameter AXI_WIDTH_DA = 32,
13     parameter AXI_WIDTH_DS = AXI_WIDTH_DA/8,
14     parameter OUT_BITS_TRANS = 18,
15     parameter WBUF_AW = 9,
16     parameter WBUF_DW = 8*3*3*16,
17     parameter WBUF_DS = WBUF_DW/8,
18     parameter MEM_BASE_ADDR = 'h8000_0000,
19     parameter MEM_DATA_BASE_ADDR = 4096
20 )
21 (
22     input                clk
23     , input              rstn
24
25     , input [31:0] i_ctrl_reg0 // network_start, //
26     , input [31:0] i_ctrl_reg1 // Read address base
27     , input [31:0] i_ctrl_reg2 // Write address base
28     , input [31:0] i_ctrl_reg3 // Write address base
29
30     , output                M_ARVALID
31     , input                M_ARREADY
32     , output [AXI_WIDTH_AD-1:0] M_ARADDR
33     , output [AXI_WIDTH_ID-1:0] M_ARID
34     , output [7:0]           M_ARLEN
35     , output [2:0]           M_ARSIZE
36     , output [1:0]           M_ARBURST
37     , output [1:0]           M_ARLOCK
38     , output [3:0]           M_ARCACHE
39     , output [2:0]           M_ARPROT
40     , output [3:0]           M_ARQOS
```

```
60     , output [2:0]           M_AWPROT
61     , output [3:0]           M_AWQOS
62     , output [3:0]           M_AWREGION
63     , output [3:0]           M_AWUSER
64
65     , output                M_WVALID
66     , input                M_WREADY
67     , output [AXI_WIDTH_DA-1:0] M_WDATA
68     , output [AXI_WIDTH_DS-1:0] M_WSTRB
69     , output                M_WLAST
70     , output [AXI_WIDTH_ID-1:0] M_WID
71     , output [3:0]           M_WUSER
72
73     , input                M_BVALID
74     , output                M_BREADY
75     , input [1:0]           M_BRESP
76     , input [AXI_WIDTH_ID-1:0] M_BID
77     , input                M_BUSER
78
79     , output network_done
80     , output network_done_led
81     // For Debugging
82     , output [AXI_WIDTH_DA-1:0] read_data
83     , output                read_data_vld
84 );
85 `include "user_define_h.v"
86 `include "user_param_h.v"
87
```

Top module (yolo_engine.v)

- Control signals (Simplified)
 - ap_start:
 - If the first bit of the control register 0 is TURN ON
 - Start the engine
 - ap_ready: Default value is 1
 - network_done
 - It is triggered when the network finishes
 - ap_done = 1

```
//-----  
// Control signals  
//-----  
always @ (*) begin  
    ap_done    = ctrl_write_done;  
    ap_ready    = 1;  
end  
assign network_done    = interrupt;  
assign network_done_led = interrupt;  
  
always @ (posedge clk, negedge rstn) begin  
    if(~rstn) begin  
        ap_start <= 0;  
    end  
    else begin  
        if(!ap_start && i_ctrl_reg0[0])  
            ap_start <= 1;  
        else if (ap_done)  
            ap_start <= 0;  
    end  
end  
  
always @(posedge clk, negedge rstn) begin  
    if(~rstn) begin  
        interrupt <= 0;  
    end  
    else begin  
        if(i_ctrl_reg0[0])  
            interrupt <= 0;  
        else if (ap_done)  
            interrupt <= 1;  
    end  
end
```

Top module (yolo_engine.v):

- Parse the configurations from the external
 - For example, the **base address** to read/write the external memory.

```
153 // Parse the control registers
154 always @ (posedge clk, negedge rstn) begin
155     if(~rstn) begin
156         dram_base_addr_rd <= 0;
157         dram_base_addr_wr <= 0;
158         reserved_register <= 0; // unused
159     end
160     else begin
161         if(!ap_start && i_ctrl_reg0[0]) begin
162             dram_base_addr_rd <= i_ctrl_reg1; // Base Address for READ (Input image, Model parameters)
163             dram_base_addr_wr <= i_ctrl_reg2; // Base Address for WRITE (Intermediate feature maps, Outputs)
164             reserved_register <= i_ctrl_reg3; // reserved
165         end
166         else if (ap_done) begin
167             dram_base_addr_rd <= 0;
168             dram_base_addr_wr <= 0;
169             reserved_register <= 0;
170         end
171     end
172 end
```

Top module: DMA read

- The flow includes:

1. Receive a READ request signal from u_dma_ctrl (ctrl_read, num_trans, read_addr)
2. Send the read request to the external memory via an AXI interface
3. Receive the return data via an AXI interface
4. Send the return data to a buffer
 - For example, request 16 x 32-bit (num_trans = 16), (e.g., 16th data)

Address
Channel

Data
Channel

User

```
u_dma_read(  
    //AXI Master Interface  
    //Read address channel  
    .M_ARVALID (M_ARVALID ), // address/control valid handshake  
    .M_ARREADY (M_ARREADY ), // Read addr ready  
    .M_ARADDR (M_ARADDR ), // Address Read  
    .M_ARID (M_ARID ), // Read addr ID  
    .M_ARLEN (M_ARLEN ), // Transfer length  
    .M_ARSIZE (M_ARSIZE ), // Transfer width  
    .M_ARBURST (M_ARBURST ), // Burst type  
    .M_ARLOCK (M_ARLOCK ), // Atomic access information  
    .M_ARCACHE (M_ARCACHE ), // Cachable/bufferable infor  
    .M_ARPROT (M_ARPROT ), // Protection info  
    .M_ARQOS (M_ARQOS ), // Quality of Service  
    .M_ARREGION (M_ARREGION ), // Region signaling  
    .M_ARUSER (M_ARUSER ), // User defined signal  
  
    //Read data channel  
    .M_RVALID (M_RVALID ), // Read data valid  
    .M_RREADY (M_RREADY ), // Read data ready (to Slave)  
    .M_RDATA (M_RDATA ), // Read data bus  
    .M_RLAST (M_RLAST ), // Last beat of a burst transfer  
    .M_RID (M_RID ), // Read ID  
    .M_RUSER (M_RUSER ), // User defined signal  
    .M_RRESP (M_RRESP ), // Read response  
  
    //Functional Ports  
    .start_dma (ctrl_read ),  
    .num_trans (num_trans ), //Number of 128-bit words transferred  
    .start_addr (read_addr ), //iteration_num * 4 * 16 + read_address_d  
    .data_o (read_data ),  
    .data_vld_o (read_data_vld),  
    .data_cnt_o (read_data_cnt),  
    .done_o (read_done ),
```

DMA read

■ Functional Ports of axi_dma_rd

■ Input

- start_dma // start the DMA read operation
- num_trans // number of transfer
- start_addr // the address from which to start the read operation

■ Output

- data_o // read data
- data_vld_o // valid signal
- data_cnt_o // data count
- done_o // done

```
u_dma_read(  
    //AXI Master Interface  
    //Read address channel  
    .M_ARVALID (M_ARVALID ), // address/control valid handshake  
    .M_ARREADY (M_ARREADY ), // Read addr ready  
    .M_ARADDR (M_ARADDR ), // Address Read  
    .M_ARID (M_ARID ), // Read addr ID  
    .M_ARLEN (M_ARLEN ), // Transfer length  
    .M_ARSIZE (M_ARSIZE ), // Transfer width  
    .M_ARBURST (M_ARBURST ), // Burst type  
    .M_ARLOCK (M_ARLOCK ), // Atomic access information  
    .M_ARCACHE (M_ARCACHE ), // Cachable/bufferable infor  
    .M_ARPROT (M_ARPROT ), // Protection info  
    .M_ARQOS (M_ARQOS ), // Quality of Service  
    .M_ARREGION (M_ARREGION ), // Region signaling  
    .M_ARUSER (M_ARUSER ), // User defined signal  
  
    //Read data channel  
    .M_RVALID (M_RVALID ), // Read data valid  
    .M_RREADY (M_RREADY ), // Read data ready (to Slave)  
    .M_RDATA (M_RDATA ), // Read data bus  
    .M_RLAST (M_RLAST ), // Last beat of a burst transfer  
    .M_RID (M_RID ), // Read ID  
    .M_RUSER (M_RUSER ), // User defined signal  
    .M_RRESP (M_RRESP ), // Read response  
  
    //Functional Ports  
    .start_dma (ctrl_read ),  
    .num_trans (num_trans ), //Number of 128-bit words transferred  
    .start_addr (read_addr ), //iteration_num * 4 * 16 + read_address_d  
    .data_o (read_data ),  
    .data_vld_o (read_data_vld),  
    .data_cnt_o (read_data_cnt),  
    .done_o (read_done ),
```


DMA write

■ The flow includes:

1. Receive a WRITE request signal from u_dma_ctrl (ctrl_write, num_trans, write_addr)
2. Send the write request to the external memory via an AXI interface
3. Send the data via an AXI interface
4. Send the return data to a buffer
 - write_data 32-bit data
 - indata_req_wr 1-bit request enable to u_data_buffer
 - write_done Mark the last data (e.g., 16th data)

Address
Channel

Data
Channel

Response
Channel

User

```
u_dma_write(  
    .M_AWADDR (M_AWADDR ), // Address Write  
    .M_AWLEN  (M_AWLEN  ), // Transfer length  
    .M_AWSIZE  (M_AWSIZE ), // Transfer width  
    .M_AWBURST (M_AWBURST ), // Burst type  
    .M_AWLOCK  (M_AWLOCK ), // Atomic access information  
    .M_AWCACHE (M_AWCACHE ), // Cachable/bufferable infor  
    .M_AWPROT  (M_AWPROT ), // Protection info  
    .M_AWREGION (M_AWREGION ),  
    .M_AWQOS   (M_AWQOS   ),  
    .M_AWVALID (M_AWVALID ), // address/control valid handshake  
    .M_AWREADY (M_AWREADY ),  
    .M_AWUSER  (           ),  
    //Write data channel  
    .M_WID      (M_WID      ), // Write ID  
    .M_WDATA    (M_WDATA    ), // Write Data bus  
    .M_WSTRB    (M_WSTRB    ), // Write Data byte lane strobes  
    .M_WLAST    (M_WLAST    ), // Last beat of a burst transfer  
    .M_WVALID   (M_WVALID   ), // Write data valid  
    .M_WREADY   (M_WREADY   ), // Write data ready  
    .M_WUSER    (           ),  
    .M_BUSER    (           ),  
    //Write response channel  
    .M_BID      (M_BID      ), // buffered response ID  
    .M_BRESP    (M_BRESP    ), // Buffered write response  
    .M_BVALID   (M_BVALID   ), // Response info valid  
    .M_BREADY   (M_BREADY   ), // Response info ready (to slave)  
    //Read address channel  
    //User interface  
    .start_dma (ctrl_write ),  
    .num_trans (num_trans  ), //Number of words transferred  
    .start_addr (write_addr ),  
    .indata     (write_data ),  
    .indata_req_o(indata_req_wr ),  
    .done_o     (write_done ), //Blk transfer done
```


DMA write

■ Functional Ports of axi_dma_wr

■ Input

- start_dma // start the DMA write operation
- num_trans // number of transfer
- mem_start_addr // the address from which to start the write operation

■ Output

- done_o // done
- indata // data to be written
- indata_req_o // data request
- fail_check // indicates write fail

```
u_dma_write(  
    .M_AWADDR (M_AWADDR ), // Address Write  
    .M_AWLEN  (M_AWLEN  ), // Transfer length  
    .M_AWSIZE  (M_AWSIZE ), // Transfer width  
    .M_AWBURST (M_AWBURST ), // Burst type  
    .M_AWLOCK  (M_AWLOCK ), // Atomic access information  
    .M_AWCACHE (M_AWCACHE ), // Cachable/bufferable infor  
    .M_AWPROT  (M_AWPROT ), // Protection info  
    .M_AWREGION (M_AWREGION ),  
    .M_AWQOS   (M_AWQOS   ),  
    .M_AWVALID (M_AWVALID ), // address/control valid handshake  
    .M_AWREADY (M_AWREADY ),  
    .M_AWUSER  (          ),  
    //Write data channel  
    .M_WID     (M_WID     ), // Write ID  
    .M_WDATA   (M_WDATA   ), // Write Data bus  
    .M_WSTRB   (M_WSTRB   ), // Write Data byte lane strobes  
    .M_WLAST   (M_WLAST   ), // Last beat of a burst transfer  
    .M_WVALID  (M_WVALID  ), // Write data valid  
    .M_WREADY  (M_WREADY  ), // Write data ready  
    .M_WUSER   (          ),  
    .M_BUSER   (          ),  
    //Write response channel  
    .M_BID     (M_BID     ), // buffered response ID  
    .M_BRESP   (M_BRESP   ), // Buffered write response  
    .M_BVALID  (M_BVALID  ), // Response info valid  
    .M_BREADY  (M_BREADY  ), // Response info ready (to slave)  
    //Read address channel  
    //User interface  
    .start_dma (ctrl_write ),  
    .num_trans (num_trans  ), //Number of words transferred  
    .start_addr (write_addr ),  
    .indata     (write_data ),  
    .indata_req_o(indata_req_wr ),  
    .done_o     (write_done ), //Blk transfer done
```

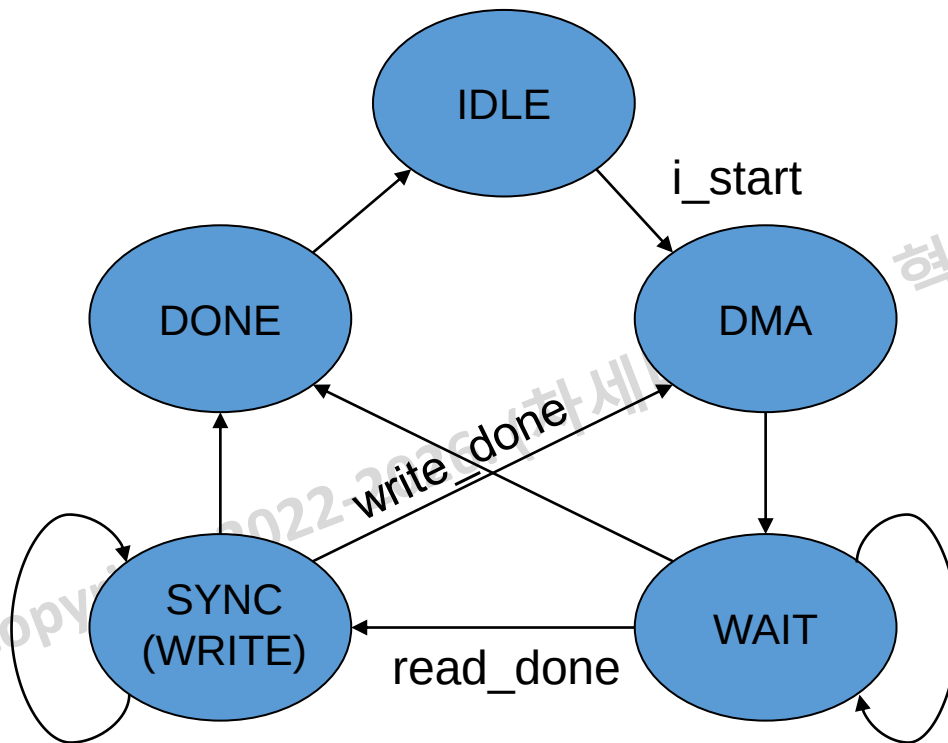
DMA Controller

- **DMA Controller: Request Generation**
- **Inputs**
 - **DRAM base addresses** for Read/Write (dram_base_addr_rd, dram_base_addr_wr)
 - **Number of transactions** for a request (num_trans). WE FIXED IT TO 16 HERE.
 - **Maximum requests** (max_req_blk_idx).
 - For example, it is $(256*256)/16$ to read an image
- **Outputs:**
 - **Send trigger signals** for read/write (ctrl_read/ctrl_write) to u_dma_read (axi_dma_rd) and u_dma_write (axi_dma_wr)

```
// DMA Controller
axi_dma_ctrl #(.BIT_TRANS(BIT_TRANS))
u_dma_ctrl(
    .clk          (clk          )
    ,.rstn        (rstn        )
    ,.i_start     (i_ctrl_reg0[0] )
    ,.i_base_address_rd(dram_base_addr_rd)
    ,.i_base_address_wr(dram_base_addr_wr)
    ,.i_num_trans (num_trans    )
    ,.i_max_req_blk_idx(max_req_blk_idx )
    // DMA Read
    ,.i_read_done (read_done    )
    ,.o_ctrl_read (ctrl_read    )
    ,.o_read_addr (read_addr    )
    // DMA Write
    ,.i_indata_req_wr (indata_req_wr )
    ,.i_write_done (write_done    )
    ,.o_ctrl_write (ctrl_write    )
    ,.o_write_addr (write_addr    )
    ,.o_write_data_cnt (write_data_cnt )
    ,.o_ctrl_write_done(ctrl_write_done )
);
```

DMA Controller (Read FSM)

- Read FSM: Generate AXI DMA read request
 - ST_DMA_WAIT:
 - Keep waiting until receiving the last beat
 - read_done = 1



```
81 always @(*) begin
82     ctrl_read = 0;
83     ctrl_read_wait = 0;
84     ctrl_read_sync = 0;
85     ctrl_read_done = 0;
86     nstate_rd = cstate_rd;
87     case(cstate_rd)
88     ST_IDLE: begin
89         if(i_start)
90             nstate_rd = ST_DMA;
91         else
92             nstate_rd = ST_IDLE;
93     end
94     ST_DMA: begin
95         nstate_rd = ST_DMA_WAIT;
96         ctrl_read = 1;
97     end
98     ST_DMA_WAIT: begin
99         ctrl_read_wait = 1;
100         if(read_done) begin
101             if (req_blk_idx_rd == max_req_blk_idx - 1)
102                 nstate_rd = ST_DMA_DONE;
103             else
104                 nstate_rd = ST_DMA_SYNC;
105         end
106     end
107     ST_DMA_SYNC: begin
108         ctrl_read_sync = 1;
109         if(write_done) begin // FIX ME
110             nstate_rd = ST_DMA;
111         end
112     end
113     ST_DMA_DONE: begin
114         ctrl_read_done = 1;
115         nstate_rd = ST_IDLE;
116     end
117 endcase
118 end
```

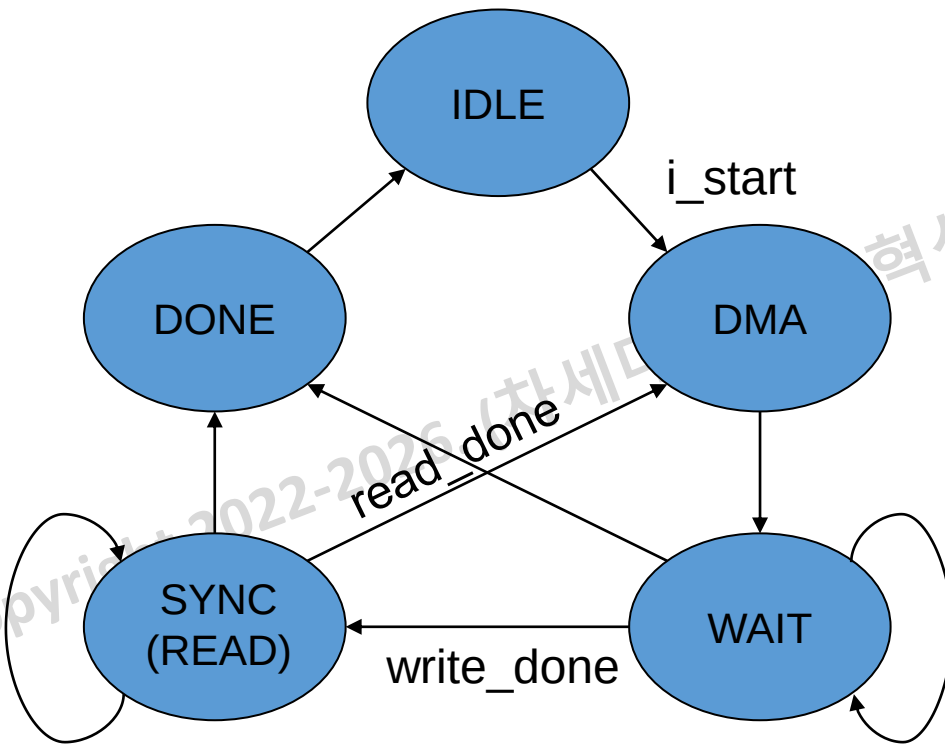
DMA Controller (Read FSM)

- Read FSM: Generate AXI DMA read request
 - Read address $\leftarrow$ Base + Offset
 - Offset $\leftarrow$ req_blk_idx_rd * 64
 - 64 = 16 * 4 (num_trans = 16, each has four bytes)

```
120 v always @(posedge clk, negedge rstn) begin
121 v     if(~rstn) begin
122 v         req_blk_idx_rd <= 0;
123 v     end
124 v     else begin
125 v         if(read_done) begin
126 v             if(req_blk_idx_rd == max_req_blk_idx - 1)
127 v                 req_blk_idx_rd <= 0;           // Reset the counter
128 v             else
129 v                 req_blk_idx_rd <= req_blk_idx_rd + 1; // Up-Counter
130 v         end
131 v     end
132 v end
133
134 assign read_addr = dram_base_addr_rd + {req_blk_idx_rd,6'b0};
```

DMA Controller (Write FSM)

- **Write FSM: Generate AXI DMA Write request**
 - ST_DMA_WAIT:
 - Keep waiting until receiving the last beat
 - read_done = 1



```

148 ~ always @(*) begin
149     ctrl_write = 0;
150     ctrl_write_wait = 0;
151     ctrl_write_sync = 0;
152     ctrl_write_done = 0;
153     nstate_wr = cstate_wr;
154 ~     case(cstate_wr)
155 ~         ST_IDLE: begin
156 ~             if(read_done) // FIX ME
157 ~                 nstate_wr = ST_DMA;
158 ~             else
159 ~                 nstate_wr = ST_IDLE;
160 ~         end
161 ~         ST_DMA: begin
162 ~             nstate_wr = ST_DMA_WAIT;
163 ~             ctrl_write = 1;
164 ~         end
165 ~         ST_DMA_WAIT: begin
166 ~             ctrl_write_wait = 1;
167 ~             if(write_done) begin
168 ~                 if (req_blk_idx_wr == max_req_blk_idx - 1)
169 ~                     nstate_wr = ST_DMA_DONE;
170 ~                 else
171 ~                     nstate_wr = ST_DMA_SYNC;
172 ~             end
173 ~         end
174 ~         ST_DMA_SYNC: begin
175 ~             ctrl_write_sync = 1;
176 ~             if(read_done) begin // FIX ME
177 ~                 nstate_wr = ST_DMA;
178 ~             end
179 ~         end
180 ~         ST_DMA_DONE: begin
181 ~             ctrl_write_done = 1;
182 ~             nstate_wr = ST_IDLE;
183 ~         end
184 ~     endcase
185 ~ end

```

DMA Controller (Write FSM)

- Write FSM: Generate AXI DMA write request
 - Write address $\leftarrow$ Base + block_offset + data_offset
 - block_offset $\leftarrow$ block_idx * 64
 - data_offset $\leftarrow$ data_idx * 4

```
187 always @(posedge clk, negedge rstn) begin
188     if(~rstn) begin
189         req_blk_idx_wr <= 0;
190     end
191     else begin
192         if(write_done) begin
193             if(req_blk_idx_wr == max_req_blk_idx - 1)
194                 req_blk_idx_wr <= 0; // Reset the counter
195             else
196                 req_blk_idx_wr <= req_blk_idx_wr + 1; // Up-Counter
197         end
198     end
199 end

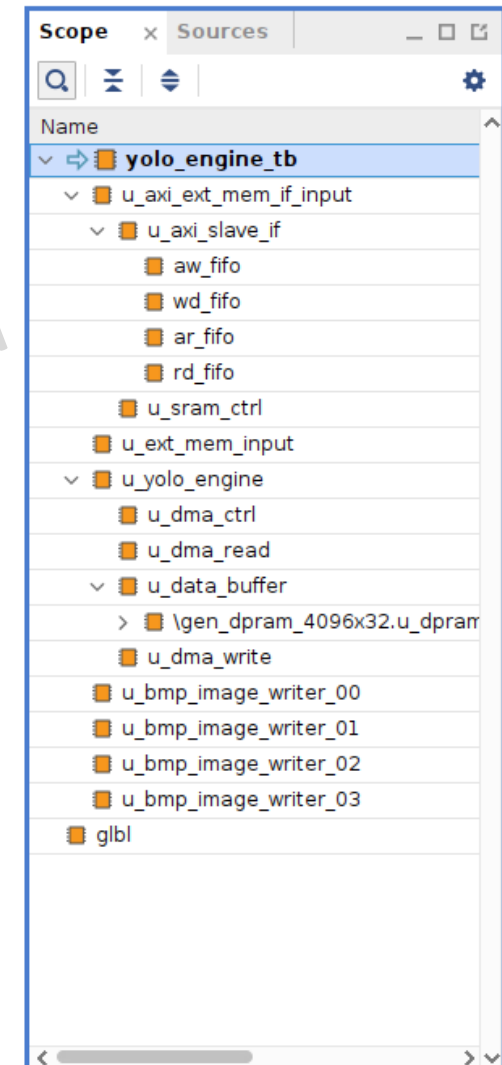
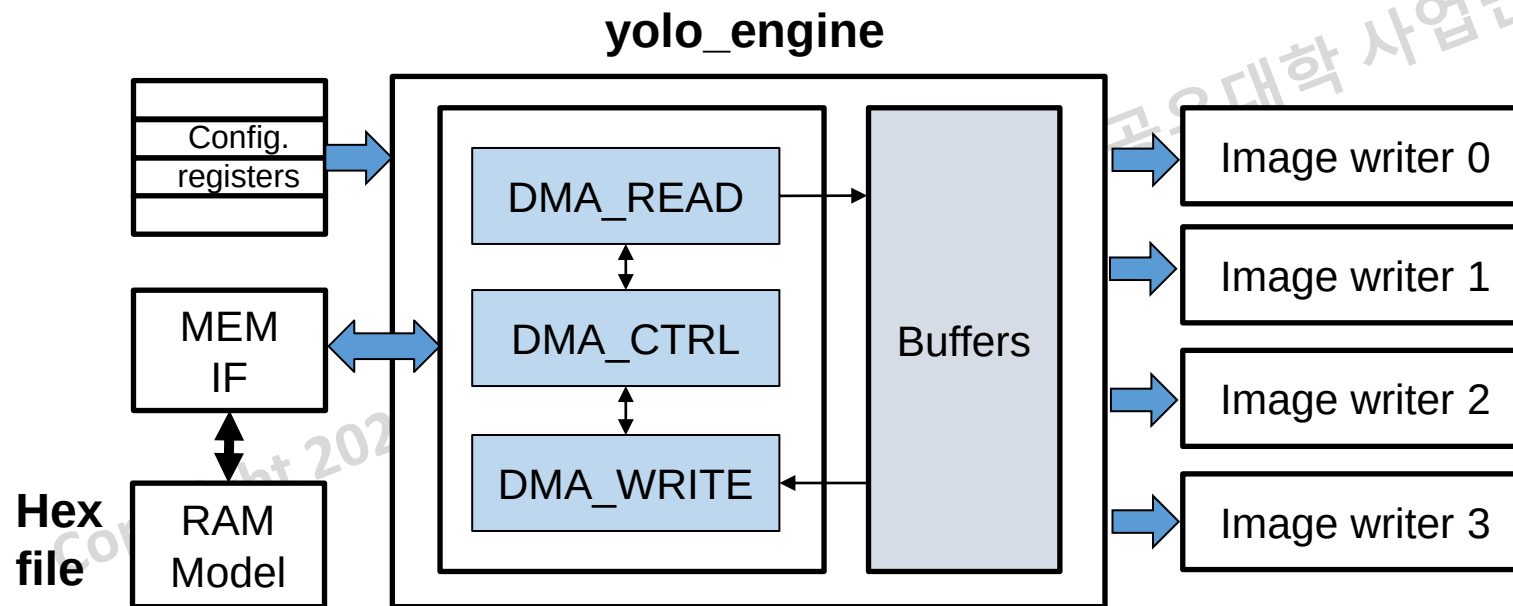
200
201 always @(posedge clk, negedge rstn) begin
202     if(~rstn) begin
203         write_data_cnt <= 0;
204     end
205     else begin
206         if(ctrl_write)
207             write_data_cnt <= 0;
208         else if (indata_req_wr) begin
209             if(write_data_cnt == num_trans - 1)
210                 write_data_cnt <= 0;
211             else
212                 write_data_cnt <= write_data_cnt + 1;
213         end
214     end
215 end

216
217 assign write_addr = dram_base_addr_wr + {req_blk_idx_wr,6'b0} + {write_data_cnt,2'b0};
```

Test bench (sim/yolo_engine_tb.v)

- This tests a simplified engine

- The engine has a DMA master and a DMA controller
- The engine is connected to a memory interface via AXI ports
- The RAM model: Memory cells are initialized from hex files



Test bench (sim/yolo_engine_tb.v)

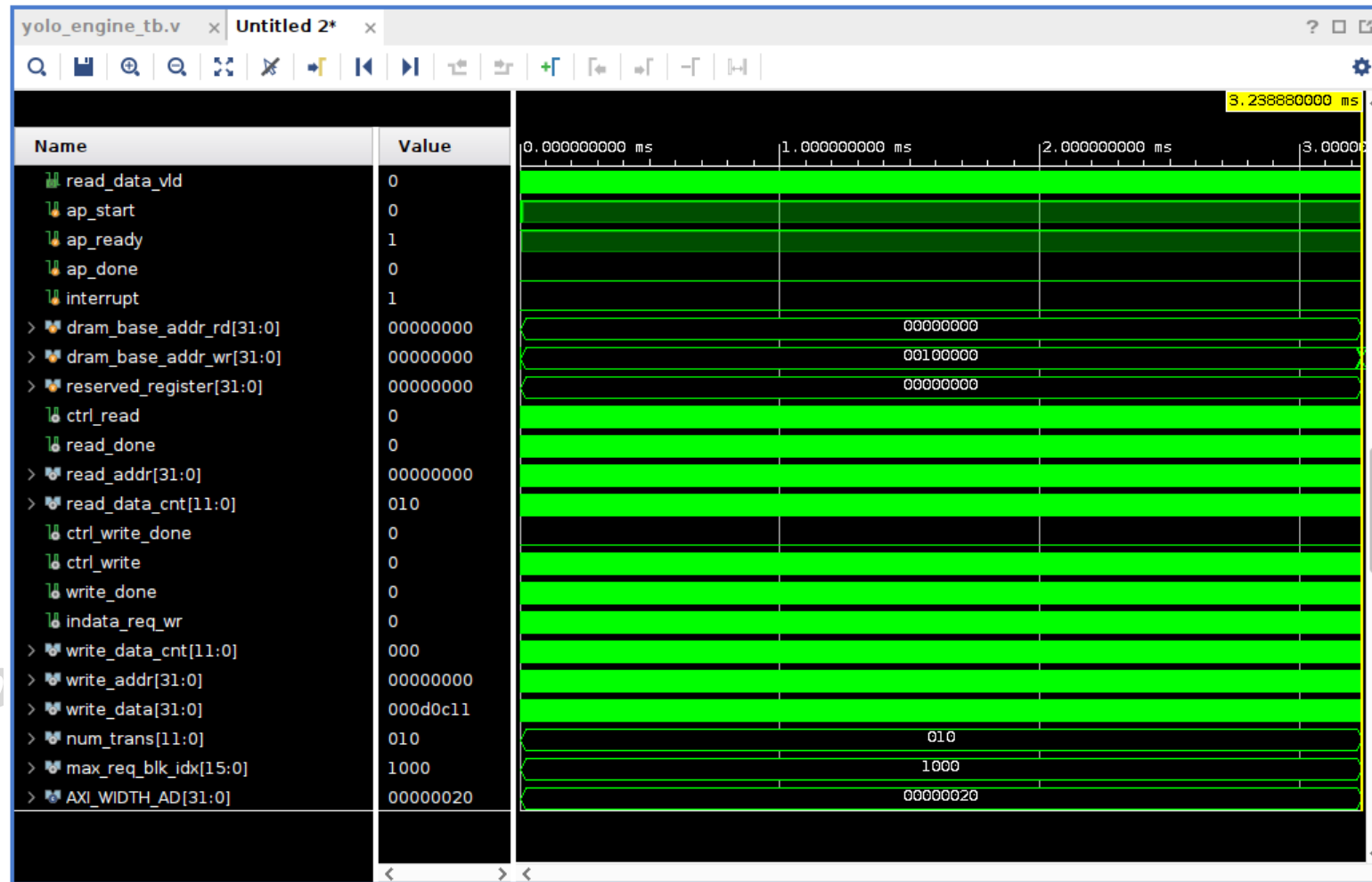
■ We can control and monitor the CNN accelerator

- ✓ Start: $i_0 \leftarrow 1$
- ✓ Send other configurations as you want
 - ✓ i_1, i_2
- ✓ Preload:
 - ✓ You may test a particular layer
 - ✓ For example, compare the results from HW with that from the software.
- ✓ Wait until the layer/network completes

```
223 initial begin
224     rstn = 1'b0;           // Reset, low active
225     i_0 = 0;
226     i_1 = 0;
227     i_2 = (4*256*256)*4;
228 `ifdef DEBUG
229     resume_counter = 0;
230 `endif
231 `ifdef PRELOAD
232     preload = 1'b0;
233     preload_layer_idx = 4;
234 `endif
235
236     #(4*CLK_PERIOD) rstn = 1'b1;
237     #(100*CLK_PERIOD)
238     @(posedge clk)
239         i_0 = 32'd1;
240     `ifdef PRELOAD
241         preload <= 1'b1;
242         preload_layer_idx <= 4;
243     `endif
244     #(100*CLK_PERIOD)
245     @(posedge clk)
246         i_0 = 32'd0;
247     while(!network_done) begin
248         #(128*CLK_PERIOD) @(posedge clk);
249     end
250     #(100*CLK_PERIOD)
251     @(posedge clk) $stop;
252 end
```

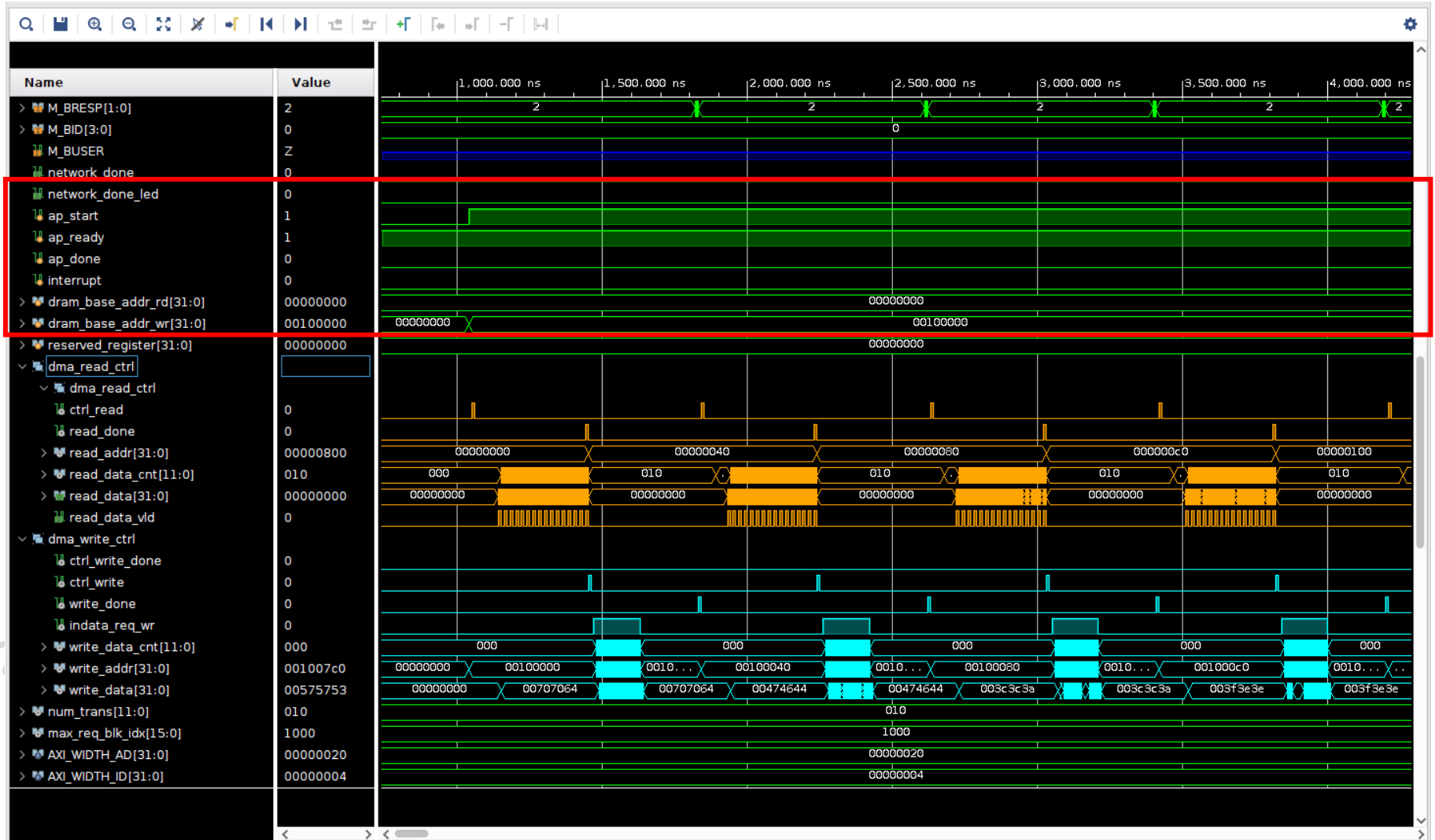
Test bench (sim/yolo_engine_tb.v)

- Simulate the top design with time = 4ms



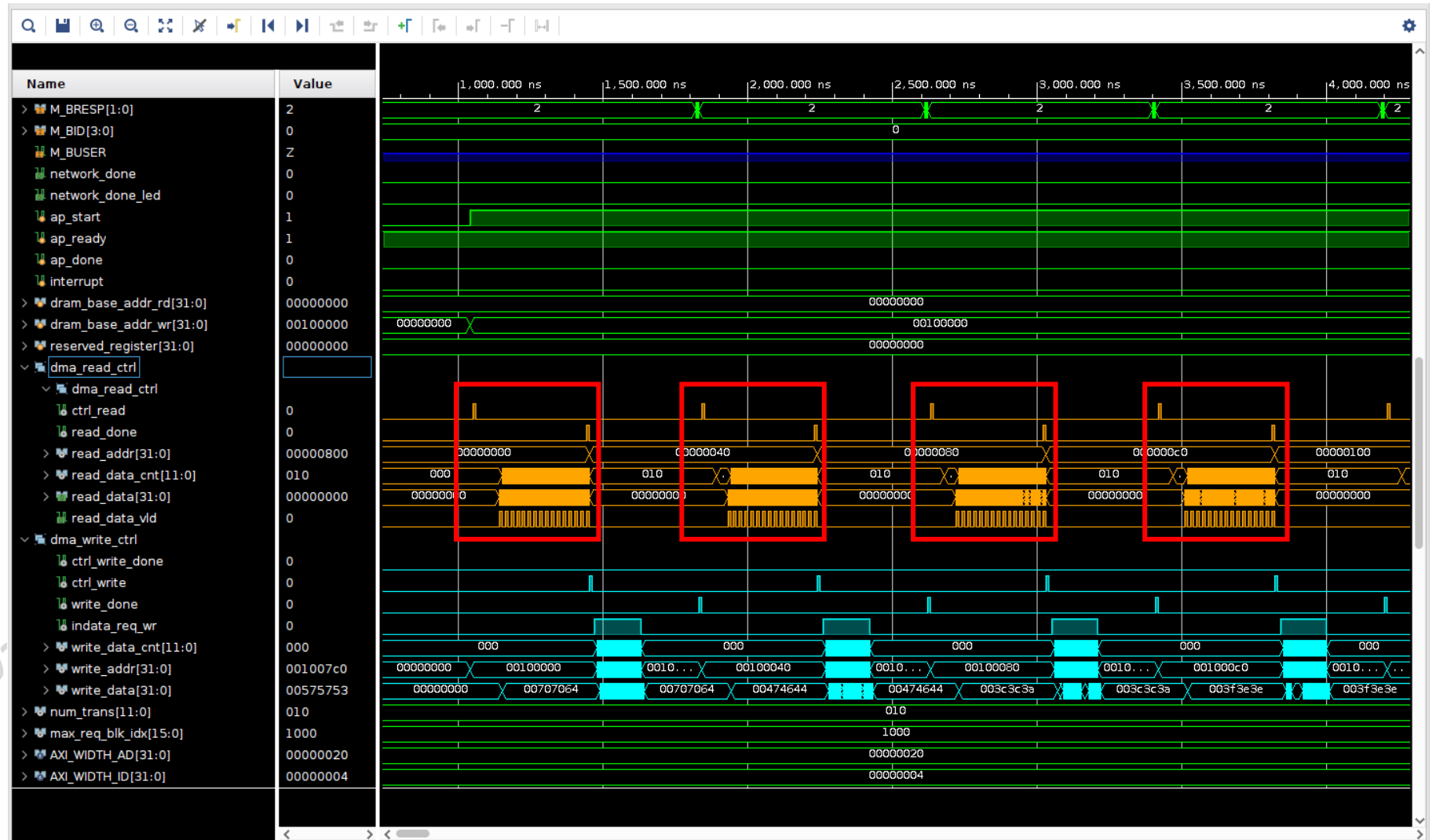
Test bench (sim/yolo_engine_tb.v)

+ Start
+ Base address
for DMA read
and write



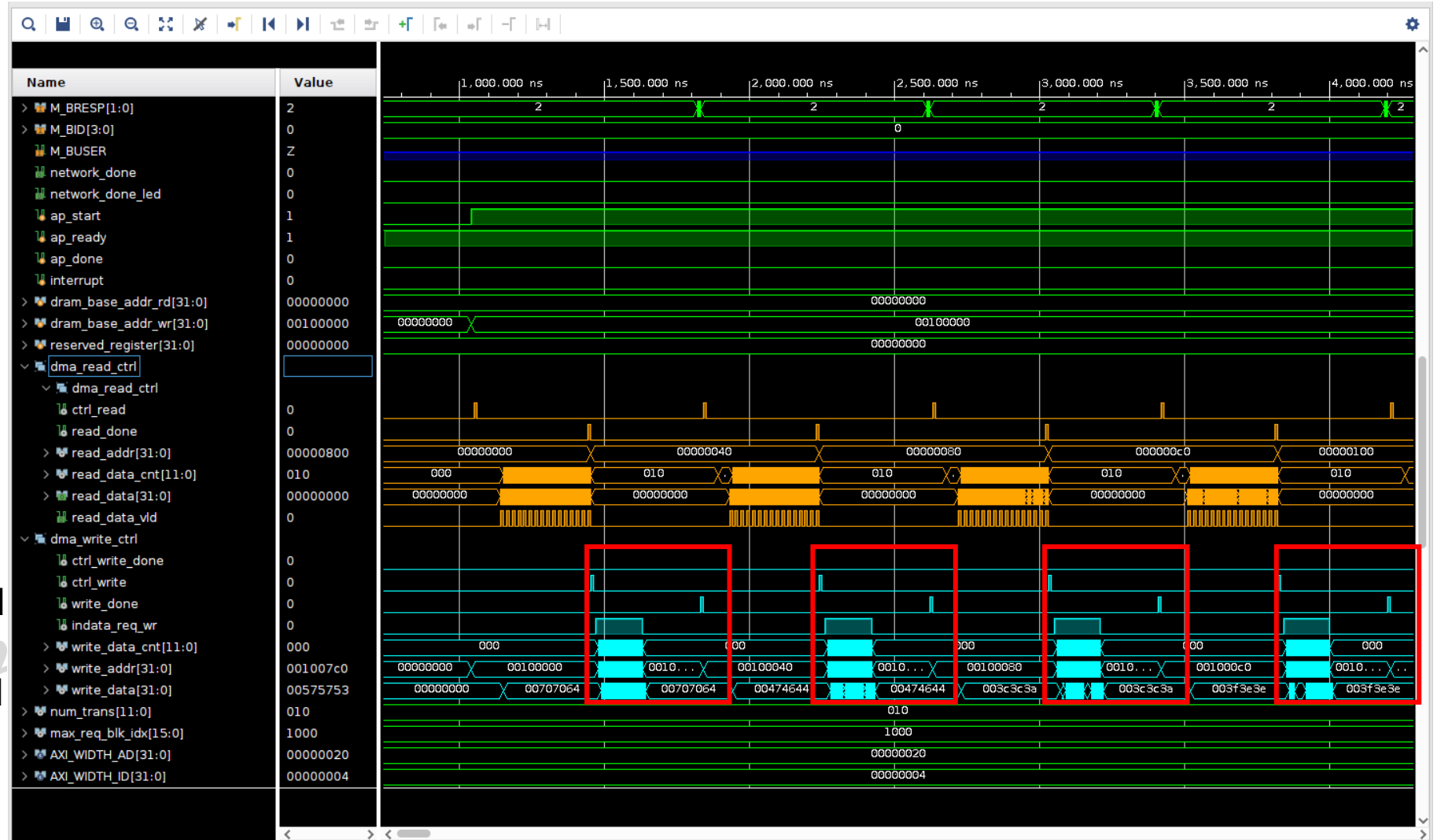
Test bench (sim/yolo_engine_tb.v)

- + DMA Read
- + Readout data are stored the Buffer
- + Address is updated



Test bench (sim/yolo_engine_tb.v)

- + DMA Write
- + Write data are read from the Buffer
- + Address is updated



Test bench (sim/yolo_engine_tb.v)

- We can save the input in BMP images

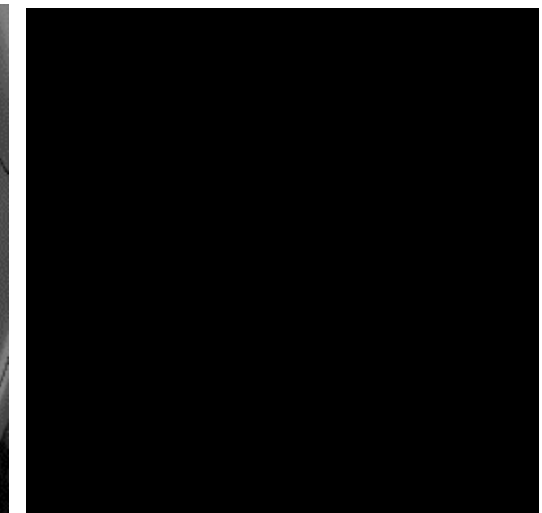
```
295 bmp_image_writer #(.OUTFILE(CONV_INPUT_IMG00),.WIDTH(IFM_WIDTH),.HEIGHT(IFM_HEIGHT))
296 u_bmp_image_writer_00(
297     /*input      */clk (clk          ),
298     /*input      */rstn(rstn         ),
299     /*input [WI-1:0] */din (read_data[7:0] ),
300     /*input      */vld (read_data_vld ),
301     /*output reg  */frame_done(      )
302 );
303 bmp_image_writer #(.OUTFILE(CONV_INPUT_IMG01),.WIDTH(IFM_WIDTH),.HEIGHT(IFM_HEIGHT))
304 u_bmp_image_writer_01(
305     /*input      */clk (clk          ),
306     /*input      */rstn(rstn         ),
307     /*input [WI-1:0] */din (read_data[15:8] ),
308     /*input      */vld (read_data_vld ),
309     /*output reg  */frame_done(      )
310 );
311 bmp_image_writer #(.OUTFILE(CONV_INPUT_IMG02),.WIDTH(IFM_WIDTH),.HEIGHT(IFM_HEIGHT))
312 u_bmp_image_writer_02(
313     /*input      */clk (clk          ),
314     /*input      */rstn(rstn         ),
315     /*input [WI-1:0] */din (read_data[23:16]),
316     /*input      */vld (read_data_vld ),
317     /*output reg  */frame_done(      )
318 );
319 bmp_image_writer #(.OUTFILE(CONV_INPUT_IMG03),.WIDTH(IFM_WIDTH),.HEIGHT(IFM_HEIGHT))
320 u_bmp_image_writer_03(
321     /*input      */clk (clk          ),
322     /*input      */rstn(rstn         ),
323     /*input [WI-1:0] */din (read_data[31:24]),
324     /*input      */vld (read_data_vld ),
325     /*output reg  */frame_done(      )
326 );
```



Test bench (sim/yolo_engine_tb.v)

- We can save the output in BMP images

```
260 `ifdef CHECK_DMA_WRITE
261   bmp_image_writer #(.OUTFILE(CONV_OUTPUT_IMG00),.WIDTH(IFM_WIDTH),.HEIGHT(IFM_HEIGHT))
262   u_bmp_image_writer_00(
263     /*input      */clk (clk          ),
264     /*input      */rstn(rstn         ),
265     /*input [WI-1:0] */din (i_WDATA[7:0] ),
266     /*input      */vld (i_WVALID      ),
267     /*output reg  */frame_done(      )
268   );
269   bmp_image_writer #(.OUTFILE(CONV_OUTPUT_IMG01),.WIDTH(IFM_WIDTH),.HEIGHT(IFM_HEIGHT))
270   u_bmp_image_writer_01(
271     /*input      */clk (clk          ),
272     /*input      */rstn(rstn         ),
273     /*input [WI-1:0] */din (i_WDATA[15:8] ),
274     /*input      */vld (i_WVALID      ),
275     /*output reg  */frame_done(      )
276   );
277   bmp_image_writer #(.OUTFILE(CONV_OUTPUT_IMG02),.WIDTH(IFM_WIDTH),.HEIGHT(IFM_HEIGHT))
278   u_bmp_image_writer_02(
279     /*input      */clk (clk          ),
280     /*input      */rstn(rstn         ),
281     /*input [WI-1:0] */din (i_WDATA[23:16] ),
282     /*input      */vld (i_WVALID      ),
283     /*output reg  */frame_done(      )
284   );
285   bmp_image_writer #(.OUTFILE(CONV_OUTPUT_IMG03),.WIDTH(IFM_WIDTH),.HEIGHT(IFM_HEIGHT))
286   u_bmp_image_writer_03(
287     /*input      */clk (clk          ),
288     /*input      */rstn(rstn         ),
289     /*input [WI-1:0] */din (i_WDATA[31:24] ),
290     /*input      */vld (i_WVALID      ),
291     /*output reg  */frame_done(      )
292   );
```



Note:

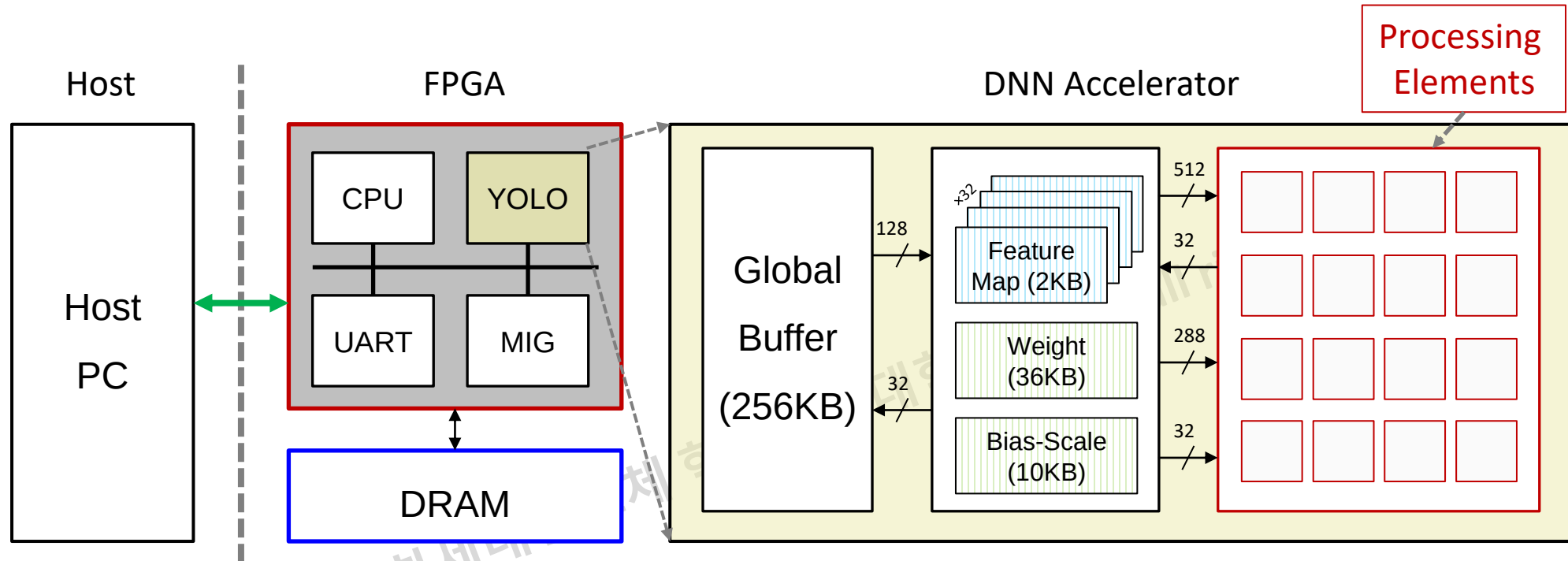
- Reproduce the result
 - Generate the dual-port DRAM model (dpram_4096x32, depth=4096, width=32, delay=1)
 - Change the paths if needed

```
src > user_param_h.v
21
22 parameter IFM_FILE_32      = "C:/yolohw/sim/inout_data_sw/log_feamap/CONV00_input_32b.hex";
23 parameter IFM_FILE        = "C:/yolohw/sim/inout_data_sw/log_feamap/CONV00_input_16b.hex";
24 parameter WGT_FILE         = "C:/yolohw/sim/inout_data_sw/log_param/CONV00_param_weight.hex";
25
26 // Output Files
27 parameter CONV_INPUT_IMG00 = "C:/yolohw/sim/inout_data_hw/CONV00_input_ch00.bmp";
28 parameter CONV_INPUT_IMG01 = "C:/yolohw/sim/inout_data_hw/CONV00_input_ch01.bmp";
29 parameter CONV_INPUT_IMG02 = "C:/yolohw/sim/inout_data_hw/CONV00_input_ch02.bmp";
30 parameter CONV_INPUT_IMG03 = "C:/yolohw/sim/inout_data_hw/CONV00_input_ch03.bmp";
31
32 parameter CONV_OUTPUT_IMG00 = "C:/yolohw/sim/inout_data_hw/CONV00_output_ch00.bmp";
33 parameter CONV_OUTPUT_IMG01 = "C:/yolohw/sim/inout_data_hw/CONV00_output_ch01.bmp";
34 parameter CONV_OUTPUT_IMG02 = "C:/yolohw/sim/inout_data_hw/CONV00_output_ch02.bmp";
35 parameter CONV_OUTPUT_IMG03 = "C:/yolohw/sim/inout_data_hw/CONV00_output_ch03.bmp";
36
37 // SRAM Size
38 parameter DW      = 32;    // data bit-width per word
39 parameter AW      = 16;    // address bit-width
40 parameter DEPTH    = 65536; // depth, word length
41 parameter N_DELAY  = 1;    // delay for spram read operation
42
43 parameter BUFF_WIDTH  = 32;
44 parameter BUFF_DEPTH  = 4096;
45 parameter BUFF_ADDR_W = $clog2(BUFF_DEPTH);
46 //}}}
```

Outlines

- Recap
- Top Module
- Multipliers and MACs
- Buffers

Recap: System Overview (DNN Accelerator Example)



- DNN Accelerator (YOLO) includes:
 - L2 Global Buffer (256KB): Can store an input ($256 \times 256 \times 4$) or an output ($128 \times 128 \times 16$)
 - L1 Buffers: Dual F-Maps ($32 \times 2\text{KB}$), Weight ($4096 \times 72 = 36\text{KB}$), Bias-Scale (2560×32)
 - Can directly access DRAM for loading an input and weights and storing outputs.

Dual-multiplier

- Compute $w \cdot x_1$ and $w \cdot x_2$
 - Reuse w for two multiplications
 - w, x_1, x_2 : Signed numbers
- Baseline: Compute with two DSPs
 - Inputs: $A[17:0]$, $B[17:0]$, $C[47:0]$
 - A, B : Signed numbers
 - $A \leftarrow \text{weight}(w)$
 - $B \leftarrow \text{input activation}(x)$
 - Outputs: $P[47:0]$

```
labB_cnn_accel_mac > src >  mul_dual.v
1  `timescale 1ns / 1ps
2  module mul_dual(
3      input clk,
4      input [7:0] w,
5      input [7:0] x1,
6      input [7:0] x2,
7      output [15:0] y1,
8      output [15:0] y2
9  );
10 //-----
11 // TODO:
12 // 1. Comment out the following codes
13 // 2. Make your design to use a single DSP
14 //-----
15 // Comment out
16 // {{{
17 wire [17:0] dsp_A1, dsp_B1;
18 wire [47:0] dsp_P1;
19 wire [17:0] dsp_A2, dsp_B2;
20 wire [47:0] dsp_P2;
21
22 assign dsp_A1 = w [7]? {10'b11_1111_1111, w } : {10'b00_0000_0000, w };
23 assign dsp_B1 = x1[7]? {10'b11_1111_1111, x1 } : {10'b00_0000_0000, x1 };
24 assign y1      = dsp_P1[15:0];
25
26 assign dsp_A2 = w [7]? {10'b11_1111_1111, w } : {10'b00_0000_0000, w };
27 assign dsp_B2 = x2[7]? {10'b11_1111_1111, x2 } : {10'b00_0000_0000, x2 };
28 assign y2      = dsp_P2[15:0];
29
30 dsp_macro_0 u_dsp1(.CLK(clk), .A(dsp_A1), .B(dsp_B1), .C(48'b0), .P(dsp_P1));
31 dsp_macro_0 u_dsp2(.CLK(clk), .A(dsp_A2), .B(dsp_B2), .C(48'b0), .P(dsp_P2));
32 //}}}
```

Testbench (mul_dual_tb.v)

- Design Under Test (DUT): mul_dual
 - reg: clk, rstn, w, x1, x2
 - Generate stimulus for DUT
 - wire: y1, y2
 - Monitor the outputs from DUT
- Clock: 100 MHz (period = 10 ns)
 - Note: Timescale = 1ns

```
1  `timescale 1ns / 1ps
2  module mul_dual_tb;
3  reg clk;
4  reg rstn;
5  reg [7:0] w, x1, x2;
6  wire[15:0] y1;
7  wire[15:0] y2;
8  //-----
9  // DUT: multiplier
10 //-----
11 mul_dual u_mul_dual(
12     /*input          */clk(clk),
13     /*input [ 7:0] */w  (w  ),
14     /*input [ 7:0] */x1 (x1 ),
15     /*input [ 7:0] */x2 (x2 ),
16     /*output[15:0] */y1 (y1 ),
17     /*output[15:0] */y2 (y2 )
18 );
19
20 // Clock
21 parameter CLK_PERIOD = 10; //100MHz
22 initial begin
23     clk = 1'b1;
24     forever #(CLK_PERIOD/2) clk = ~clk;
25 end
```

Testbench (mul_dual_tb.v)

■ Initialization

- All are set to zeros

■ Reset: Active low (rstn: 0 → 1)

- Similar to Push a button

■ Test cases:

- for $i = 0 \rightarrow 15$

- $w \leftarrow 4$

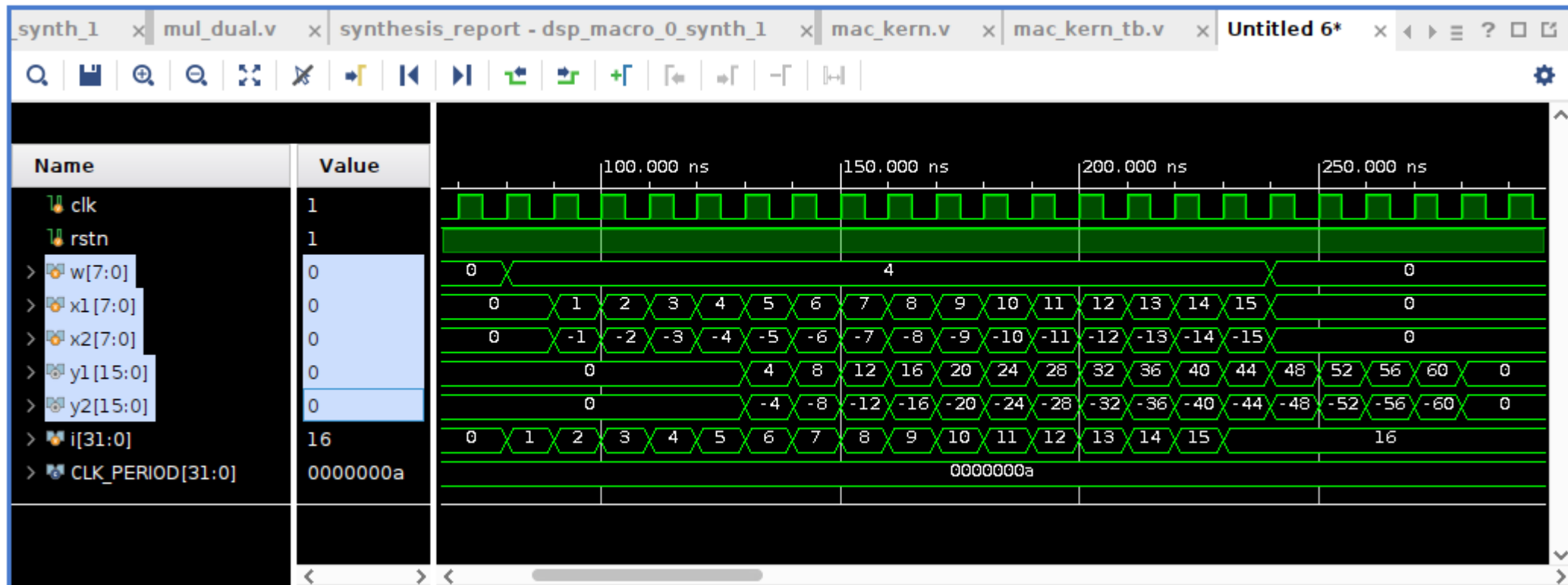
- $x1 \leftarrow i$

- $x2 \leftarrow -i$

```
26 integer i;
27 // Test cases
28 initial begin
29     rstn = 1'b0;           // Reset, low active
30     w = 0;
31     x1 = 0;
32     x2 = 0;
33     i = 0;
34     #(4*CLK_PERIOD) rstn = 1'b1;
35
36     #(4*CLK_PERIOD)
37     for(i = 0; i<16; i=i+1) begin
38         w = 8'd4;
39         x1 = i;
40         x2 = -i;
41     end
42
43     #(CLK_PERIOD)
44     w = 8'd0;
45     x1 = 8'd0;
46     x2 = 8'd0;
47 end
48 endmodule
49
```

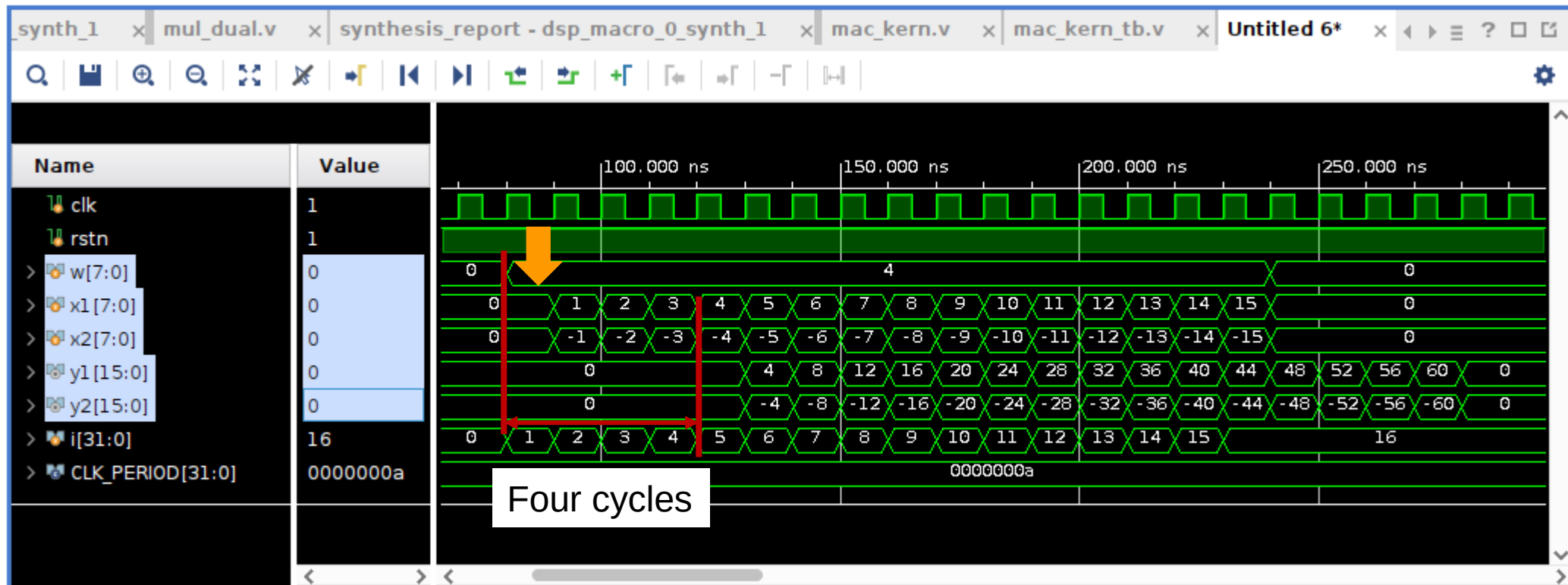

Simulation Result

- Simulation time = 300 ns
- Inputs (w, x1, x2) and Outputs (y1, y2)



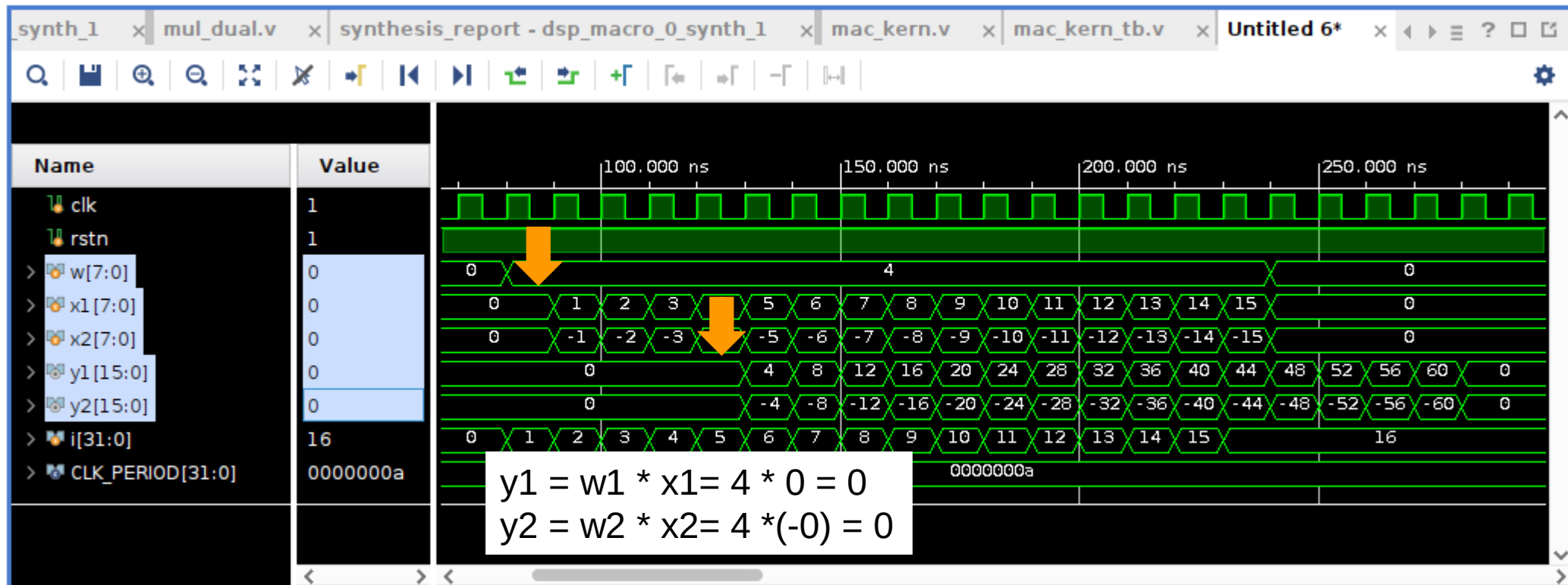
Simulation Result

- Simulation time = 300 ns
- Inputs (w, x1, x2) and Outputs (y1, y2)



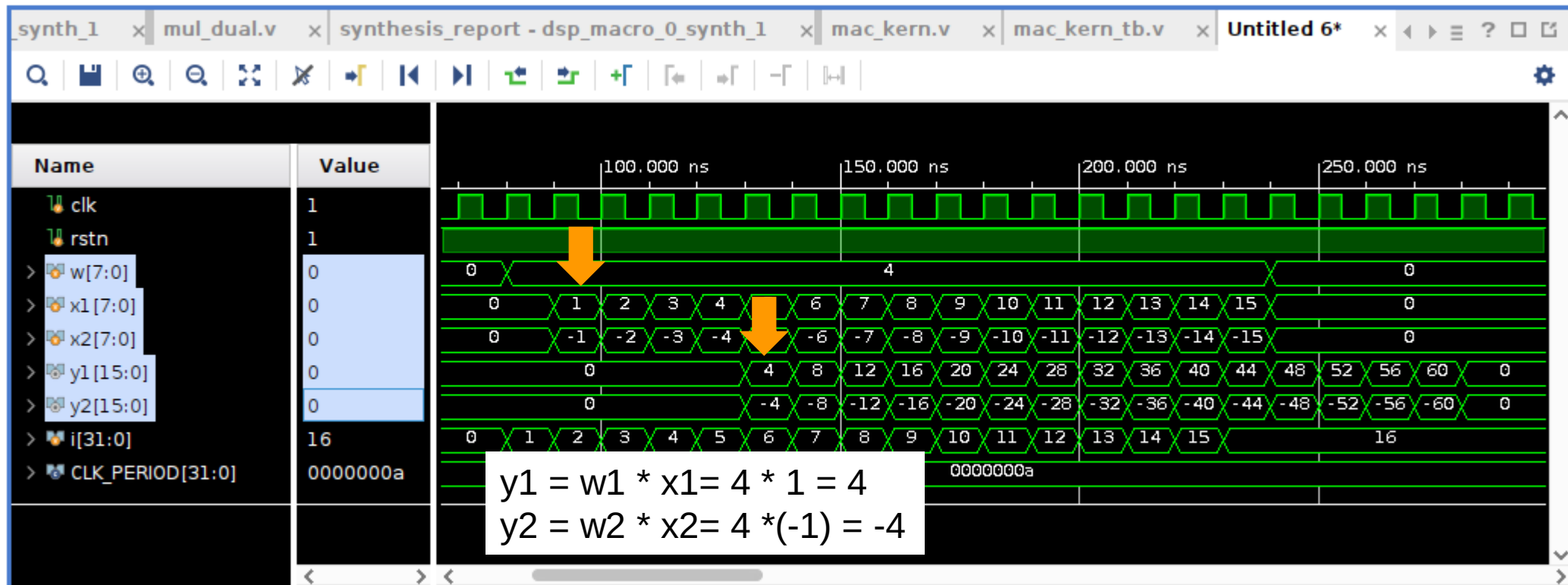
Simulation Result

- Simulation time = 300 ns
- Inputs (w, x1, x2) and Outputs (y1, y2)



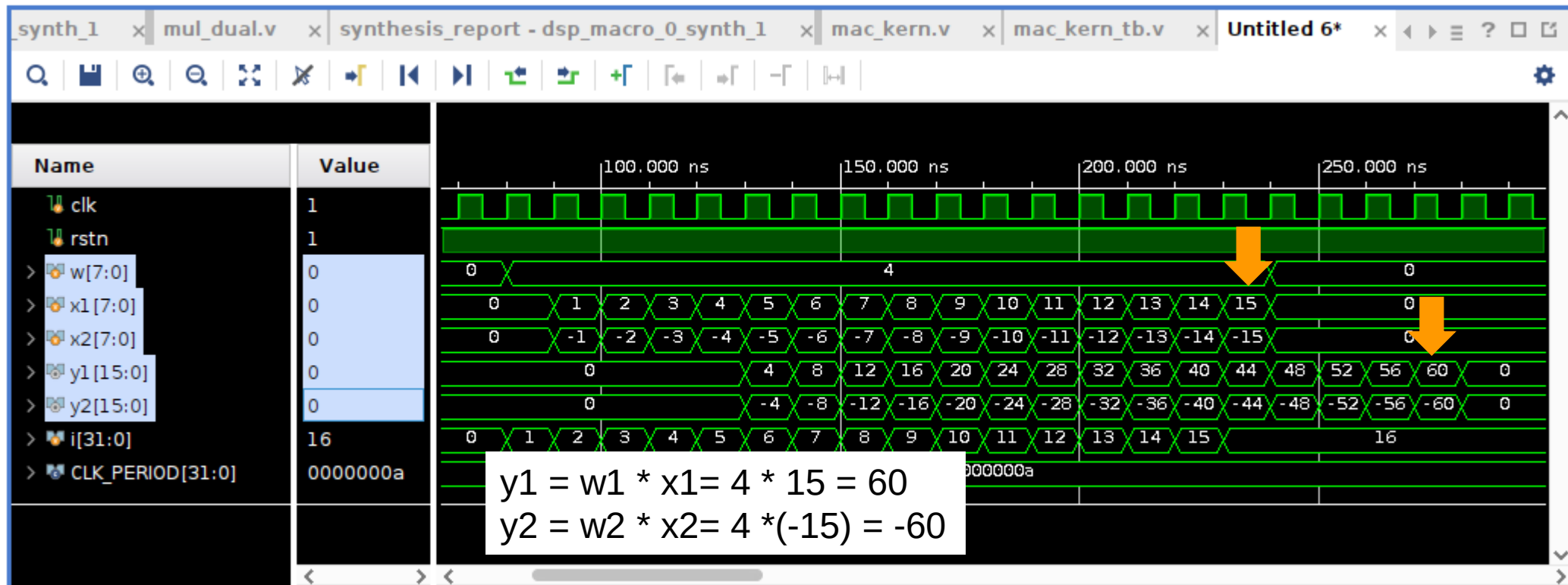
Simulation Result

- Simulation time = 300 ns
- Inputs (w, x1, x2) and Outputs (y1, y2)



Simulation Result

- Simulation time = 300 ns
- Inputs (w, x1, x2) and Outputs (y1, y2)



Multiplier Stack and Adder Trees

- **Implement a 144-MAC module**
 - Include four 36-multiplier groups
 - Use four adder trees
 - Inputs:
 - Weight (wgt): 36 ($=3 \times 3 \times 4$) eight-bit coefficients
 - IFM (ifm_00/01/10/11): 36 ($=3 \times 3 \times 4$) eight-bit activations
 - Outputs
- **Testbench**

```
labB_cnn_accel_mac > src > mac_stack.v
1  `timescale 1ns / 1ps
2  module mac_stack (
3      input      clk,
4      input      rstn,
5      input      vld_i,
6      input [36*8-1:0] wgt,
7      input [36*8-1:0] ifm_00,
8      input [36*8-1:0] ifm_01,
9      input [36*8-1:0] ifm_10,
10     input [36*8-1:0] ifm_11,
11     output [21:0] acc_00,
12     output [21:0] acc_01,
13     output [21:0] acc_10,
14     output [21:0] acc_11,
15     output      vld_o
16 );
17
18 wire [36*16-1:0] res_00;
19 wire [36*16-1:0] res_01;
20 wire [36*16-1:0] res_10;
21 wire [36*16-1:0] res_11;
22
```

MAC stacks

- Compute a sum of 36 products

$$Y = \sum_{i=0}^{3 \times 3 \times 4 - 1} w_i * x_i$$

- Pseudocode

$$y_0^{(0)} = w_0 * x_0, \dots, y_{35}^{(0)} = w_{35} * x_{35}$$

// 36 multipliers

$$y_0^{(1)} = y_0^{(0)} + y_1^{(0)} + y_2^{(0)}, \dots, y_{11}^{(1)} = y_{33}^{(0)} + y_{34}^{(0)} + y_{35}^{(0)}$$

// 12 three-input adders

$$y_0^{(2)} = y_0^{(1)} + y_1^{(1)} + y_2^{(1)}, \dots, y_3^{(2)} = y_9^{(1)} + y_{10}^{(1)} + y_{11}^{(1)}$$

// 4 three-input adders

$$y_0^{(3)} = y_0^{(2)} + y_1^{(2)}, \dots, y_1^{(3)} = y_2^{(2)} + y_3^{(2)}$$

// 2 two-input adders

$$y_0^{(4)} = y_0^{(3)} + y_1^{(3)}$$

// 1 two-input adder

$$Y = y_0^{(4)}$$

// Output

MAC stacks (mac_stack.v)

- Compute a sum of 36 products

$$Y = \sum_{i=0}^{3 \times 3 \times 4 - 1} w_i * x_i$$

- Pseudocode

$$y_0^{(0)} = w_0 * x_0, \dots, y_{35}^{(0)} = w_{35} * x_{35}$$

$$y_0^{(1)} = y_0^{(0)} + y_1^{(0)} + y_2^{(0)}, \dots, y_{11}^{(1)} = y_{33}^{(0)} + y_{34}^{(0)} + y_{35}^{(0)}$$

$$y_0^{(2)} = y_0^{(1)} + y_1^{(1)} + y_2^{(1)}, \dots, y_3^{(2)} = y_9^{(1)} + y_{10}^{(1)} + y_{11}^{(1)}$$

$$y_0^{(3)} = y_0^{(2)} + y_1^{(2)}, \dots, y_1^{(3)} = y_2^{(2)} + y_3^{(2)}$$

$$y_0^{(4)} = y_0^{(3)} + y_1^{(3)}$$

$$Y = y_0^{(4)}$$

- Have 144 multipliers (two sets of dual multipliers)
 - Multiplication results are in res00~11

```
27 // Generate for loop to instantiate 36 mul_dual modules
28 genvar i;
29 generate
30     for (i = 0; i < 36; i = i + 1) begin: mult_stack_even
31         mul_dual u_mul_dual_even(
32             /*input      *//clk (clk          ),
33             /*input [7 :0] */w   (wgt   [i* 8+: 8]),
34             /*input [7 :0] */x1  (ifm_00 [i* 8+: 8]),
35             /*input [7 :0] */x2  (ifm_01 [i* 8+: 8]),
36             /*output[15:0] */y1  (res_00 [i*16+:16]),
37             /*output[15:0] */y2  (res_01 [i*16+:16])
38         );
39     end
40 endgenerate
41
42 generate
43     for (i = 0; i < 36; i = i + 1) begin: mult_stack_odd
44         mul_dual u_mul_dual_odd(
45             /*input      *//clk (clk          ),
46             /*input [7 :0] */w   (/*Add your code*/),
47             /*input [7 :0] */x1  (/*Add your code*/),
48             /*input [7 :0] */x2  (/*Add your code*/),
49             /*output[15:0] */y1  (/*Add your code*/),
50             /*output[15:0] */y2  (/*Add your code*/)
51         );
52     end
53 endgenerate
```

MAC stacks (mac_stack.v)

- Compute a sum of 36 products

$$Y = \sum_{i=0}^{3 \times 3 \times 4 - 1} w_i * x_i$$

- Pseudocode

$$y_0^{(0)} = w_0 * x_0, \dots, y_{35}^{(0)} = w_{35} * x_{35}$$

$$y_0^{(1)} = y_0^{(0)} + y_1^{(0)} + y_2^{(0)}, \dots, y_{11}^{(1)} = y_{33}^{(0)} + y_{34}^{(0)} + y_{35}^{(0)}$$

$$y_0^{(2)} = y_0^{(1)} + y_1^{(1)} + y_2^{(1)}, \dots, y_3^{(2)} = y_9^{(1)} + y_{10}^{(1)} + y_{11}^{(1)}$$

$$y_0^{(3)} = y_0^{(2)} + y_1^{(2)}, \dots, y_1^{(3)} = y_2^{(2)} + y_3^{(2)}$$

$$y_0^{(4)} = y_0^{(3)} + y_1^{(3)}$$

$$Y = y_0^{(4)}$$

- 36 results are accumulated by an adder tree
 - Four trees are used.

```
labB_cnn_accel_mac > src > mac_stack.v
74  adder_tree_36in u_adder_tree_00(
75      /*input          */rstn   (rstn   ),
76      /*input          */vld_i   (vld_i_d4),
77      /*input [36*16-1:0] */mul_res(res_00 ),
78      /*output[ 21:0] */acc_o   (acc_00 ),
79      /*output          */vld_o   (vld[0] )
80  );
81
82
83  adder_tree_36in u_adder_tree_01(
84      /*input          */clk   (clk   ),
85      /*input          */rstn   (rstn   ),
86      /*input          */vld_i   (vld_i_d4),
87      /*input [36*16-1:0] */mul_res(res_01 ),
88      /*output[ 21:0] */acc_o   (acc_01 ),
89      /*output          */vld_o   (vld[1] )
90  );
91
92  adder_tree_36in u_adder_tree_10(
93      /*input          */clk   (clk   ),
94      /*input          */rstn   (rstn   ),
95      /*input          */vld_i   (vld_i_d4),
96      /*input [36*16-1:0] */mul_res(res_10 ),
97      /*output[ 21:0] */acc_o   (acc_10 ),
98      /*output          */vld_o   (vld[2] )
99  );
100
101  adder_tree_36in u_adder_tree_11(
102      /*input          */clk   (clk   ),
103      /*input          */rstn   (rstn   ),
104      /*input          */vld_i   (vld_i_d4),
105      /*input [36*16-1:0] */mul_res(res_11 ),
106      /*output[ 21:0] */acc_o   (acc_11 ),
107      /*output          */vld_o   (vld[3] )
108  );
```

Adder Tree (add_tree_36in.v)

Inputs:

- clk, rstn, and input valid signal (vld_i)
- multiplier results (mul_res) $\leftarrow 36 * 16$ bits

Outputs:

- Accumulated result (acc_o)
- Output valid signal (vld_o)

```
abB_cnn_accel_mac > src > adder_tree_36in.v
1  `timescale 1ns / 1ps
2
3  module adder_tree_36in(
4  input      clk,
5  input      rstn,
6  input      vld_i,
7  input [36*16-1:0] mul_res,
8  output [21:0] acc_o,
9  output      vld_o
10 );
11
```

```
12 //-----
13 // Signals
14 //-----
15 // Level 1
16 wire [17:0] y1_00;
17 wire [17:0] y1_01;
18 wire [17:0] y1_02;
19 wire [17:0] y1_03;
20 wire [17:0] y1_04;
21 wire [17:0] y1_05;
22 wire [17:0] y1_06;
23 wire [17:0] y1_07;
24 wire [17:0] y1_08;
25 wire [17:0] y1_09;
26 wire [17:0] y1_10;
27 wire [17:0] y1_11;
28
29 // Level 2
30 wire [19:0] y2_0;
31 wire [19:0] y2_1;
32 wire [19:0] y2_2;
33 wire [19:0] y2_3;
34 // Level 3
35 wire [20:0] y3_0;
36 wire [20:0] y3_1;
37 // Level 4
38 wire [21:0] y4;
39 // Delays
40 reg vld_i_d1, vld_i_d2, vld_i_d3, vld_i_d4;
```

12 results
(Layer 1)

4 results
(Layer 2)

2 results
(Layer 3)
Final result

Adder Tree (add_tree_36in.v)

- Four-level adder tree
 - Levels 1, 2: three-input adders, Levels 3, 4: two-input adders

```
44 // Level 1
45 add3 #(.W(16)) u_add3_l1_00 (.clk(clk), .in1(mul_res[ 0*16+:16]), .in2(mul_res[ 1*16+:16]), .in3(mul_res[ 2*16+:16]),
46 add3 #(.W(16)) u_add3_l1_01 (.clk(clk), .in1(mul_res[ 3*16+:16]), .in2(mul_res[ 4*16+:16]), .in3(mul_res[ 5*16+:16]),
47 add3 #(.W(16)) u_add3_l1_02 (.clk(clk), .in1(mul_res[ 6*16+:16]), .in2(mul_res[ 7*16+:16]), .in3(mul_res[ 8*16+:16]),
48 add3 #(.W(16)) u_add3_l1_03 (.clk(clk), .in1(mul_res[ 9*16+:16]), .in2(mul_res[10*16+:16]), .in3(mul_res[11*16+:16]),
49 add3 #(.W(16)) u_add3_l1_04 (.clk(clk), .in1(mul_res[12*16+:16]), .in2(mul_res[13*16+:16]), .in3(mul_res[14*16+:16]),
50 add3 #(.W(16)) u_add3_l1_05 (.clk(clk), .in1(mul_res[15*16+:16]), .in2(mul_res[16*16+:16]), .in3(mul_res[17*16+:16]),
51 add3 #(.W(16)) u_add3_l1_06 (.clk(clk), .in1(mul_res[18*16+:16]), .in2(mul_res[19*16+:16]), .in3(mul_res[20*16+:16]),
52 add3 #(.W(16)) u_add3_l1_07 (.clk(clk), .in1(mul_res[21*16+:16]), .in2(mul_res[22*16+:16]), .in3(mul_res[23*16+:16]),
53 add3 #(.W(16)) u_add3_l1_08 (.clk(clk), .in1(mul_res[24*16+:16]), .in2(mul_res[25*16+:16]), .in3(mul_res[26*16+:16]),
54 add3 #(.W(16)) u_add3_l1_09 (.clk(clk), .in1(mul_res[27*16+:16]), .in2(mul_res[28*16+:16]), .in3(mul_res[29*16+:16]),
55 // Insert your code
56 // Insert your code
57
58 // Level 2
59 add3 #(.W(18)) u_add3_l2_00 (.clk(clk), .in1(y1_00), .in2(y1_01), .in3(y1_02), .out(y2_0));
60 add3 #(.W(18)) u_add3_l2_01 (.clk(clk), .in1(y1_03), .in2(y1_04), .in3(y1_05), .out(y2_1));
61 add3 #(.W(18)) u_add3_l2_02 (.clk(clk), .in1(y1_06), .in2(y1_07), .in3(y1_08), .out(y2_2));
62 // Insert your code
63
64 // Level 3
65 add2 #(.W(20)) u_add2_l3_00 (.clk(clk), .in1(y2_0), .in2(y2_1), .out(y3_0));
66 // Insert your code
67
68 // Level 4
69 // Insert your code
```

Delays and Outputs (mac_stack.v)

- With four-level adder tree, the output appears after 4 cycles.
 - Delay signals are used to generate vld_o from vld_i (~shift register)

```
70 //-----  
71 // valid signal  
72 //-----  
73 always@(posedge clk, negedge rstn) begin  
74     if(!rstn) begin  
75         vld_i_d1 <= 0;  
76         vld_i_d2 <= 0;  
77         vld_i_d3 <= 0;  
78         vld_i_d4 <= 0;  
79     end  
80     else begin  
81         vld_i_d1 <= vld_i ;  
82         vld_i_d2 <= vld_i_d1;  
83         vld_i_d3 <= vld_i_d2;  
84         vld_i_d4 <= vld_i_d3;  
85     end  
86 end  
87 //Output  
88 assign vld_o = vld_i_d4;  
89 assign acc_o = $signed(y4);
```

Testbench (mac_stack_tb.v)

- Design Under Test (DUT): mac_stack
 - reg: clk, rstn, wgt, ifm_**, vld_i
 - Generate stimulus for DUT
 - wgt, ifm: 36×8-bits
 - wire: acc_**, vld_o
 - Monitor the outputs from DUT
 - Accumulated results: 22 bits

```
abB_cnn_accel_mac > sim > ≡ mac_stack_tb.v
1  `timescale 1ns / 1ps
2  module mac_stack_tb;
3  reg      clk;
4  reg      rstn;
5  reg      vld_i;
6  reg [36*8-1:0] wgt;
7  reg [36*8-1:0] ifm_00;
8  reg [36*8-1:0] ifm_01;
9  reg [36*8-1:0] ifm_10;
10 reg [36*8-1:0] ifm_11;
11 wire[ 21:0] acc_00;
12 wire[ 21:0] acc_01;
13 wire[ 21:0] acc_10;
14 wire[ 21:0] acc_11;
15 wire      vld_o;
16
17 //-----
18 // DUT: MAC stack
19 //-----
20 mac_stack u_mac_stack(
21  /*input      */clk    (clk    ),
22  /*input      */rstn   (rstn   ),
23  /*input      */vld_i  (vld_i  ),
24  /*input [36*8-1:0] */wgt  (wgt   ),
25  /*input [36*8-1:0] */ifm_00(ifm_00),
26  /*input [36*8-1:0] */ifm_01(ifm_01),
27  /*input [36*8-1:0] */ifm_10(ifm_10),
28  /*input [36*8-1:0] */ifm_11(ifm_11),
29  /*output[ 21:0] */acc_00(acc_00),
30  /*output[ 21:0] */acc_01(acc_01),
31  /*output[ 21:0] */acc_10(acc_10),
32  /*output[ 21:0] */acc_11(acc_11),
33  /*output      */vld_o (vld_o  ),
34 );
```

Testbench (mac_stack_tb.v)

- Clock: 100 MHz
- Initialize a data array
- Send a trigger to a counter
 - ~address generator
 - Generate a test vector

```
65 //-----
66 // Test vector generation
67 //-----
68 always@(*) begin
69     vld_i = 0;
70     wgt = 0;
71     ifm_00 = 0;
72     ifm_01 = 0;
73     ifm_10 = 0;
74     ifm_11 = 0;
75     if(run_counter) begin
76         vld_i = 1;
77         wgt = {36{8'd1}};
78         ifm_00 = {36{data_array[counter]}};
79         ifm_01 = {36{data_array[counter]}};
80         ifm_10 = {36{data_array[counter]}};
81         ifm_11 = {36{data_array[counter]}};
82     end
83 end
```

3

```
36 // Clock
37 parameter CLK_PERIOD = 10; //100MHz
38 initial begin
39     clk = 1'b1;
40     forever #(CLK_PERIOD/2) clk = ~clk;
41 end
42 reg [31:0] i;
43 reg trigger;
44 reg run_counter;
45 reg [7:0] counter;
46 reg [7:0] data_array[0:15];
47 initial begin
48     data_array[ 0] = 8'd1; data_array[ 1]
49     data_array[ 4] = 8'd5; data_array[ 5]
50     data_array[ 8] = 8'd1; data_array[ 9]
51     data_array[12] = 8'd5; data_array[13]
52 end
53 // Test cases
54 initial begin
55     rstn = 1'b0; // Reset, low active
56     i = 0;
57     trigger = 0;
58     #(4*CLK_PERIOD) rstn = 1'b1;
59     #(4*CLK_PERIOD) trigger = 0;
60     #( CLK_PERIOD) trigger = 1;
61     #( CLK_PERIOD) trigger = 0;
62 end
63 end
64

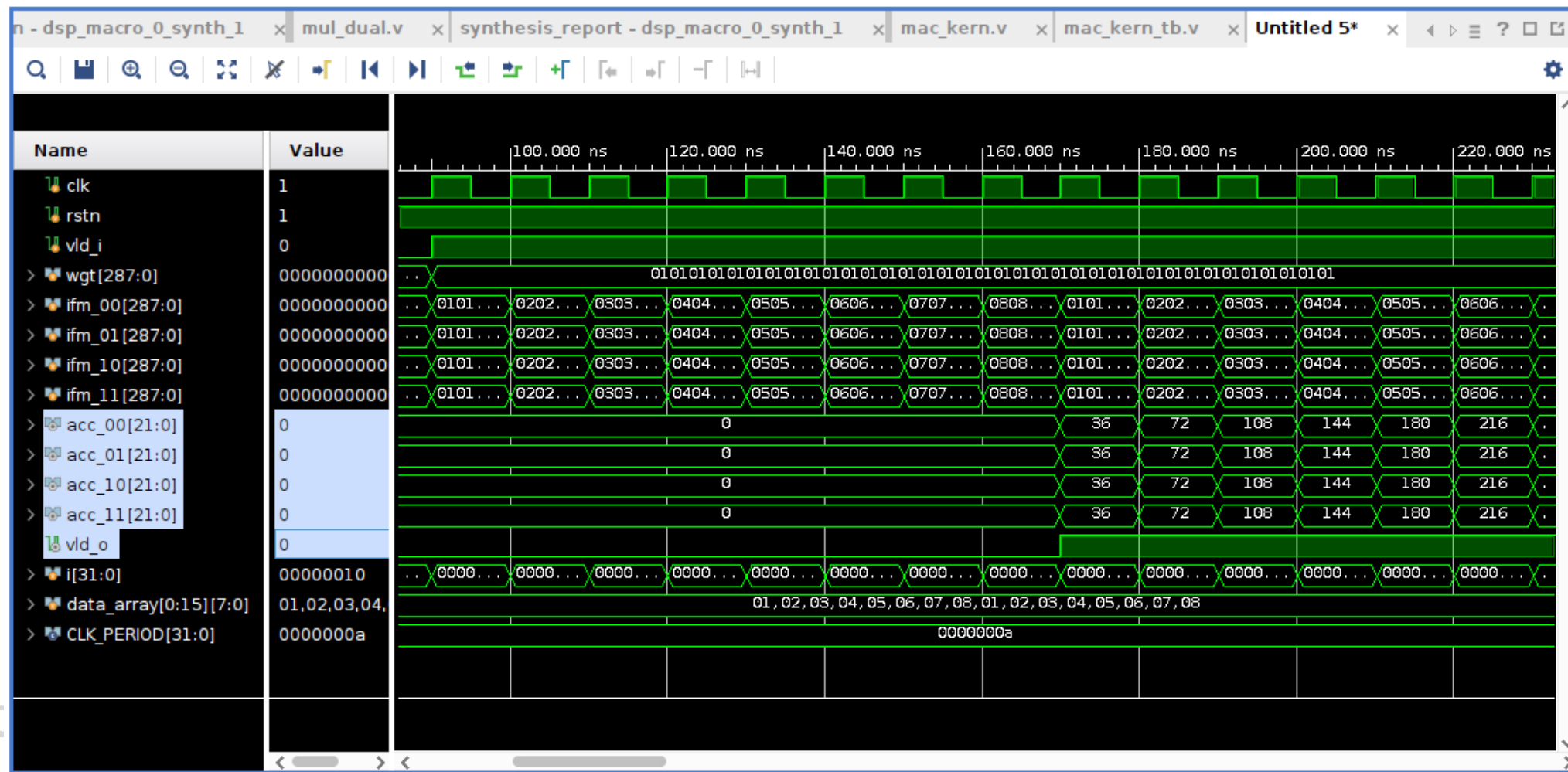
84 always @(posedge clk or negedge rstn) begin
85     if(!rstn) begin
86         run_counter <= 0;
87     end
88     else begin
89         if(trigger && (counter == 0 || counter == 15))
90             run_counter <= 1;
91         else if(counter == 15)
92             run_counter <= 0;
93     end
94 end
95

96 always @(posedge clk or negedge rstn) begin
97     if(!rstn) begin
98         counter <= 0;
99     end
100     else begin
101         if(run_counter) begin
102             if(counter == 15)
103                 counter <= 0;
104             else
105                 counter <= counter + 1;
106         end
107     end
108 end
```

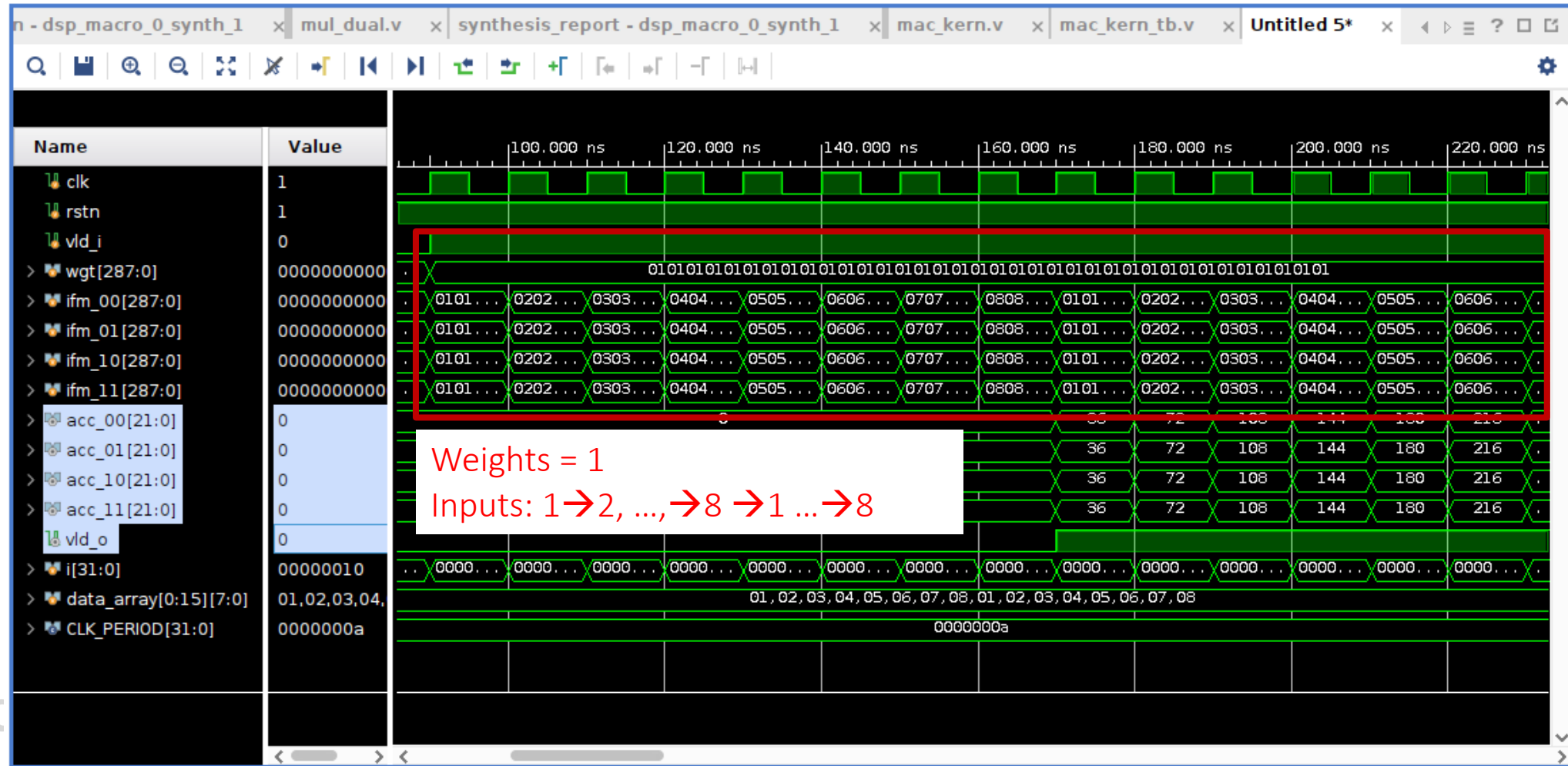
1

2

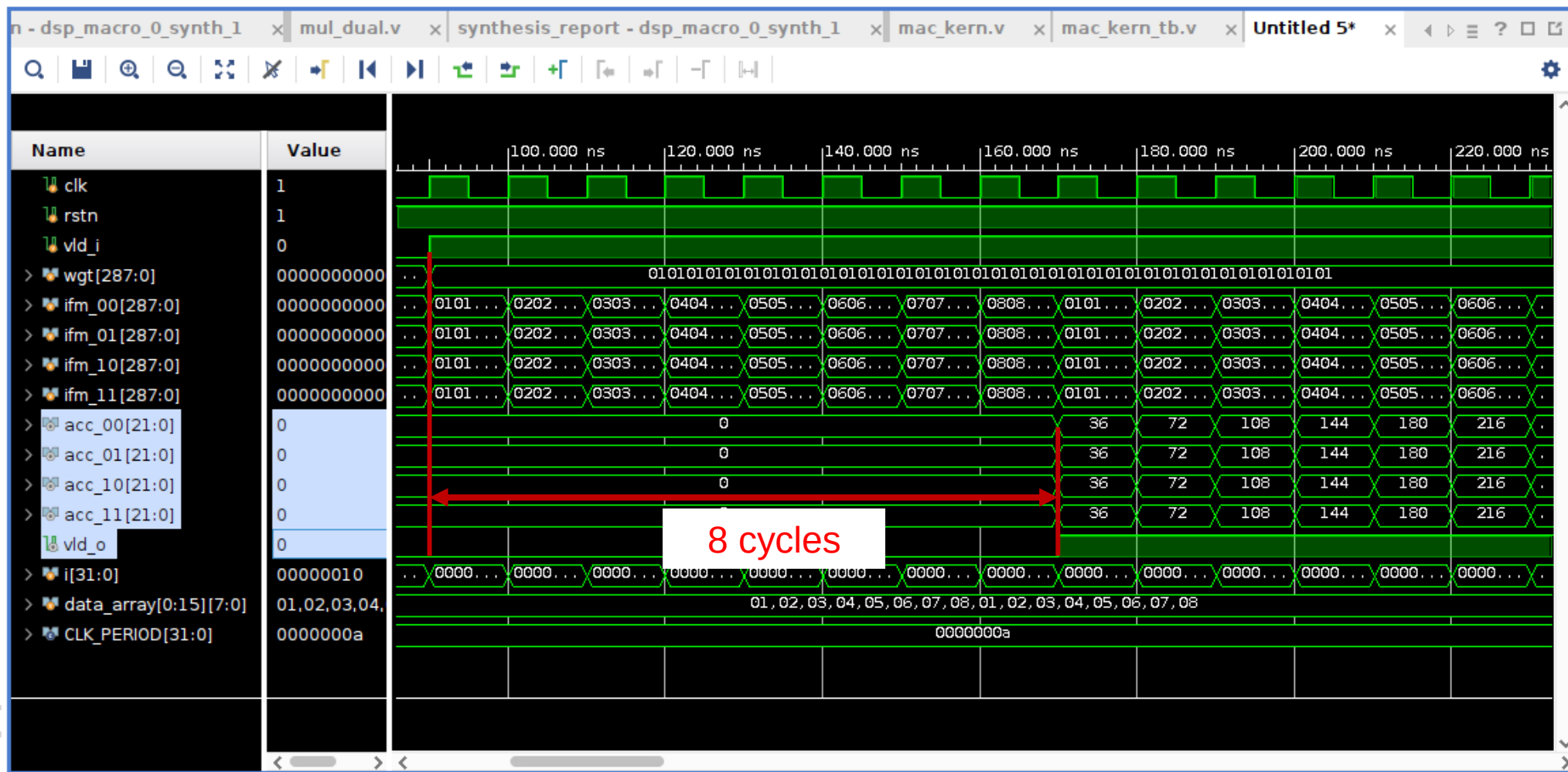
Simulation Result: Time=400ns



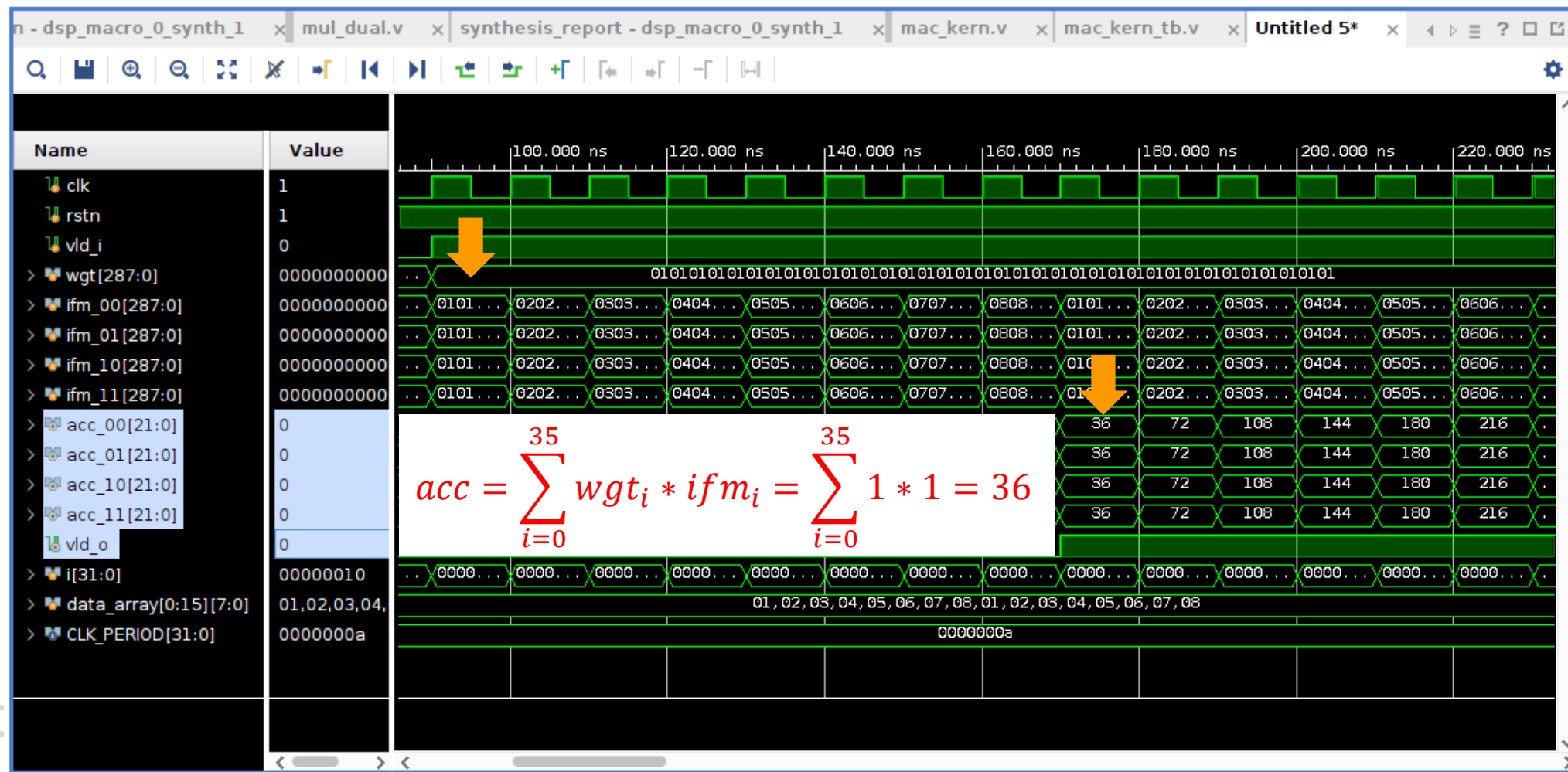
Simulation Result: Time=400ns



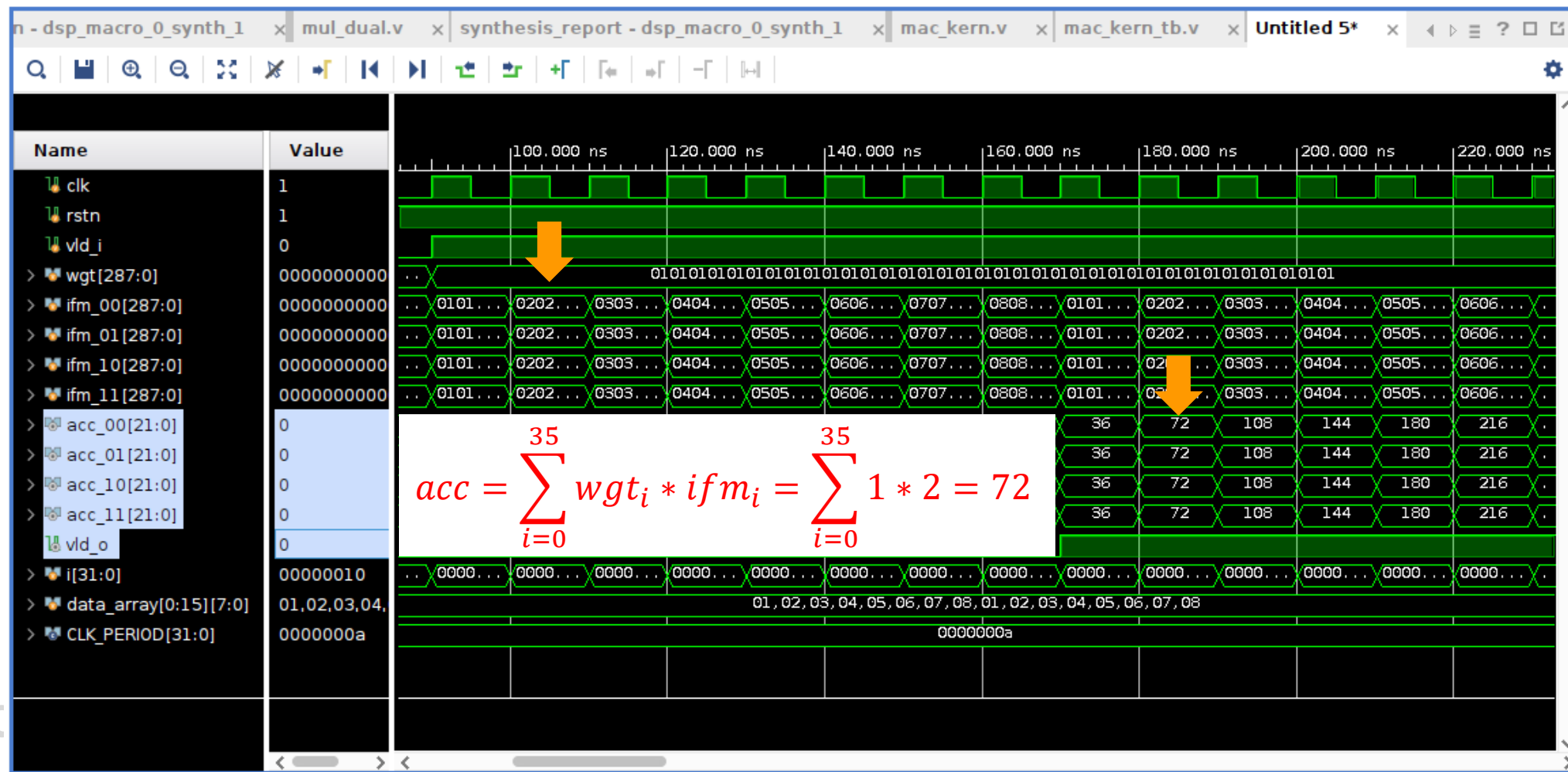
Simulation Result: Time=400ns



Simulation Result: Time=400ns



Simulation Result: Time=400ns



MAC Kernel with Accumulators

- Implement a 144-MAC module
 - Utilize mac_stack.v (144 MACs)
 - Inputs:
 - WGT: 36 ($=3 \times 3 \times 4$) eight-bit filters
 - IFM: 64 ($=4 \times 4 \times 4$) eight-bit activations
 - Len: Accumulation len
 - Mode: CONV3x3 & CONV1x1
 - Outputs
 - OFM: Accumulated output (before adding a bias and descaling)
 - Output valid
 - A request to load a bias and a scale
- Testbench

```
labB_cnn_accel_mac > src > mac_kern.v
1  `timescale 1ns / 1ps
2  module mac_kern(
3      input          clk,
4      input          rstn,
5      input          i_start,
6      input          i_mode,
7      input          i_vld,
8      input [7:0]    i_len,
9      input [36*8-1:0] i_wgt,
10     input [64*8-1:0] i_ifm,
11     output          o_req,
12     output[4*32-1:0] o_ofm,
13     output reg      o_vld
14 );
```

MAC Kernel (mac_kern.v)

- Utilize MAC stacks
 - Compute four accumulated results (acc_00~11)

```
58  always@(*) begin
59      // Dummy code
60      wgt    = i_wgt;
61      ifm_00 = i_wgt;
62      ifm_01 = i_wgt;
63      ifm_10 = i_wgt;
64      ifm_11 = i_wgt;
65      // Insert your code
66      // {{{
67      if(i_mode == 0) begin    // CONV3x3
68          // wgt    = i_wgt;
69          //ifm_00 = i_wgt;
70          //ifm_01 = i_wgt;
71          //ifm_10 = i_wgt;
72          //ifm_11 = i_wgt;
73      end
74      else begin              // CONV1x1
75          // wgt    = i_wgt;
76          //ifm_00 = i_wgt;
77          //ifm_01 = i_wgt;
78          //ifm_10 = i_wgt;
79          //ifm_11 = i_wgt;
80      end
81      // }}}
82  end
```

```
83  //=====
84  // Multiplier-Adder Stack
85  //=====
86  mac_stack u_mac_stack (
87      /*input          */clk    (clk    ),
88      /*input          */rstn    (rstn    ),
89      /*input          */vld_i   (i_vld   ),
90      /*input [36*8-1:0] */wgt    (wgt    ),
91      /*input [36*8-1:0] */ifm_00(ifm_00),
92      /*input [36*8-1:0] */ifm_01(ifm_01),
93      /*input [36*8-1:0] */ifm_10(ifm_10),
94      /*input [36*8-1:0] */ifm_11(ifm_11),
95      /*output[      21:0] */acc_00(acc_00),
96      /*output[      21:0] */acc_01(acc_01),
97      /*output[      21:0] */acc_10(acc_10),
98      /*output[      21:0] */acc_11(acc_11),
99      /*output          */vld_o   (vld_o   )
100 );
```


MAC Kernel (mac_kern.v)

- Partial sum
 - One mac stack can compute up to 36 MACs
 - This accumulated result can be added to the partial sum (psum) based on the given accumulation length (i_len or r_len)

$$psum = \sum_j \sum_i w_i * x_i$$

Computed by
mac_kern

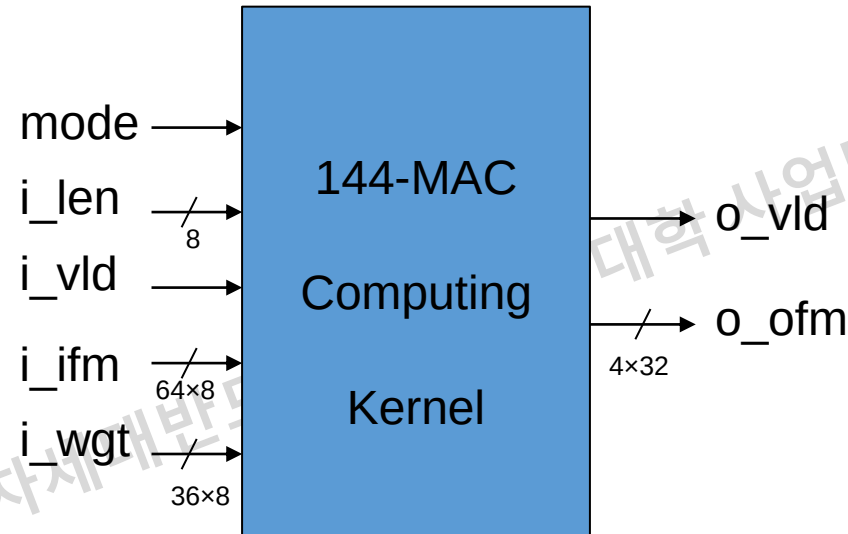
Computed by
one MAC stack

- Example: r_len = 4
 - Cycle 1 (acc_len == 0) psum = acc
 - Others (acc_len == 1, 2, 3) psum += acc

```
102 //=====
103 // Partial sum
104 //=====
105 always@(posedge clk or negedge rstn) begin
106     if(!rstn) begin
107         psum_00 <= 0;
108         psum_01 <= 0;
109         psum_10 <= 0;
110         psum_11 <= 0;
111     end
112     else begin
113         if(vld_o) begin
114             if(acc_len == 0) begin
115                 psum_00 <= acc_00;
116                 psum_01 <= 0 /*Insert your code*/;
117                 psum_10 <= 0 /*Insert your code*/;
118                 psum_11 <= 0 /*Insert your code*/;
119                 if(acc_len == r_len-1)
120                     acc_len <= 0;
121             else
122                 acc_len <= acc_len + 1;
123             end
124         else begin
125             psum_00 <= $signed(psum_00) + $signed(acc_00);
126             psum_01 <= $signed(psum_01) + $signed(acc_01);
127             psum_10 <= 0 /* Insert your code */;
128             psum_11 <= 0 /* Insert your code */;
129             if(acc_len == r_len-1)
130                 acc_len <= 0;
131             else
132                 acc_len <= acc_len + 1;
133             end
134         end
135     end
136 end
```

Summary on MACs

- A sample design with 144 MACs
 - Inputs: mode, i_len, i_vld, i_ifm, i_wgt
 - Outputs: o_vld, o_ofm



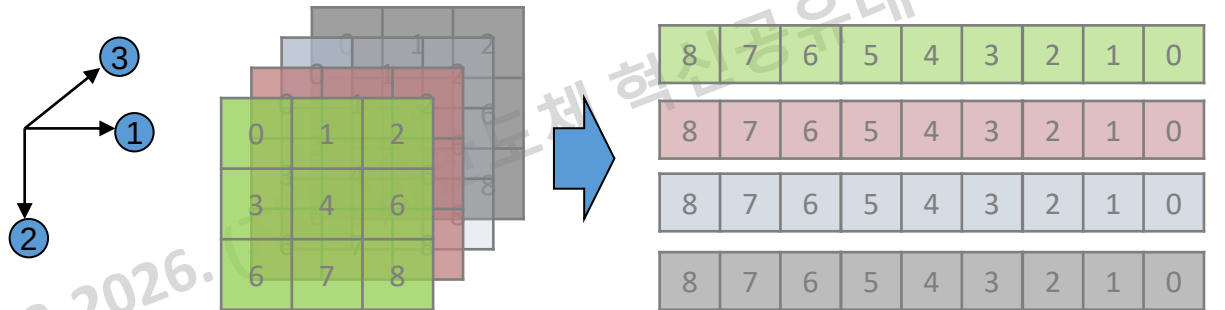
- IMPORTANT NOTE: You can have different design options,
 - Option A: $144 = (3 \times 3 \times 4) \times (2 \times 2 \times 1) \rightarrow$ Generate $2 \times 2 \times 1$ pixels, each w/ 32 MACs
 - Option B: $144 = (3 \times 3 \times 1) \times (2 \times 2 \times 4) \rightarrow$ Generate $2 \times 2 \times 4$ pixels, each w/ 9 MACs

Outlines

- Recap
- Top Module
- Multipliers and MACs
- Buffers

Example Design: Weight Buffer

- Word size: 72 bits
 - Conv3×3: A filter 3×3×1×8b
 - Conv1×1: A filter 1×1×8×8b
- A number of words: 4096



Example Design: Weight Buffer

- Word size: 72 bits
 - Conv3×3: A filter 3×3×1×8b
 - Conv1×1: A filter 1×1×8×8b
- A number of words: 4096
 - Buffer size: $4096 \times 72 = 8 \times (512 \times 72) =$ eight 36Kb BRAMs

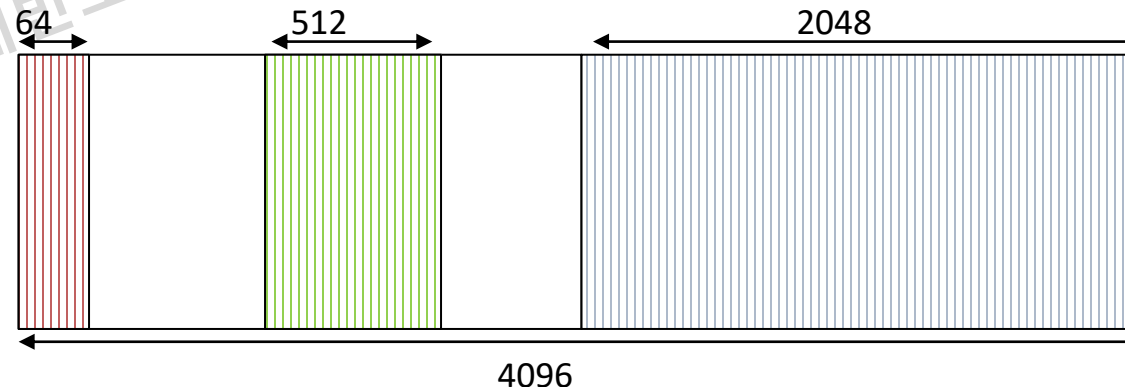
Layer	Input	Output	Weight	start_addr	Range	Remark
0	256×256×3	256×256×16	3×3×3×16	0	64 (=4*16)	ROM like
2	128×128×16	128×128×32	3×3×16×32	512	512 (=16*32)	ROM like
4	64×64×32	64×64×64	3×3×32×64	2048	2048 (=32*64)	ROM like

Example: Coefficient (COE) file

```

1 memory_initialization_radix=16;
2 memory_initialization_vector=
3 ebfe17d6ef3dfaf513,
4 fdfc0be0f230fbf411,
5 fbff06e1ef3100f010,
6 000000000000000000,
7 ef0206ee0505eff10c,
8 eae9e3f2fed4f5f0ed,
9 27240b2c3efa1a1311,
10 000000000000000000,
11 09ea051501f80e0603,
12 1403140e0610110908,
13 efd707e5d8f7e6db00,
14 000000000000000000,
15 ff09fc18170b0c1118,
16 fcfcf400f8fcf8fd14,
    
```

- Initialization
 - Layer 0: 0~63
 - Layer 1: 512~1023
 - Layer 2: 2048~4097



Example: Design weight/scale/bias buffers

- Implement a global buffer for parameters
 - Weights: 4096×72
 - Initialize Layer 0, 2, 4.
 - Bias/Scale: 2560×32
 - 16 bits for a bias and a scale
 - Store biases and scales for all layers
 - Inputs:
 - dma_wea, dma_addra, dma_dina
 - wgt_addrb, bsc_addra
 - Outputs
 - wgt_doutb
 - bsc_douta
- Testbench

```
src > gbuff_param.v
1  `timescale 1ns / 1ps
2  module gbuff_param(
3      input      clk,
4      // From DMA: Weight
5      input      dma_wea,
6      input [11:0] dma_addra,
7      input [71:0] dma_dina,
8
9      // From MAC: Weights
10     input [ 9:0] wgt_addrb,
11     output [287:0] wgt_doutb,
12     // From MAC: Bias/Scale
13     input [11:0] bsc_addra,
14     output [31:0] bsc_douta
15 );
16
```

IP Configuration: Weight Buffer

- Name: dpram_4096x72
- Dual-port RAM 4096x72
 - Simple Dual Port RAM
 - Port A: Write Only
 - Port B: Read Only
 - No. of words: 4096
 - Word size: 72

Re-customize IP@broccoli

Block Memory Generator (8.4)

Documentation IP Location Switch to Defaults

IP Symbol Power Estimation

☐ Show disabled ports

Component Name dpram_4096x72

Basic Port A Options Port B Options Other Options Summary

Interface Type Native

Memory Type Simple Dual Port RAM

☐ Generate address interface with 32 bits

☐ Common Clock

ECC Options

ECC Type No ECC

☐ Error Injection Pins Single Bit Error Injection

Write Enable

☐ Byte Write Enable

Byte Size (bits) 9

Algorithm Options

Defines the algorithm used to concatenate the block RAM primitives. Refer datasheet for more information.

Algorithm Minimum Area

Primitive 8kx2

OK Cancel

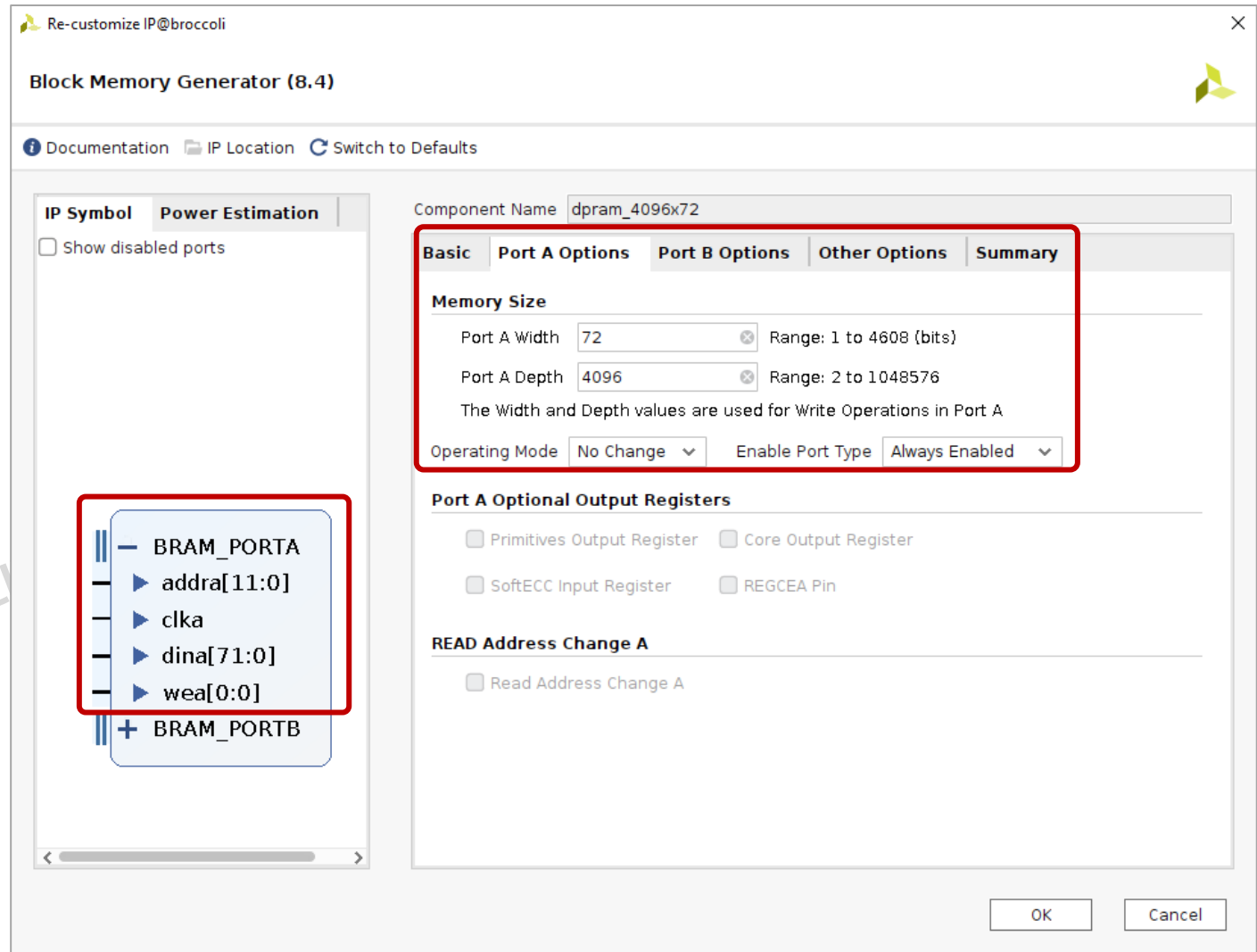
IP Configuration: Weight Buffer

■ Port A (Write Only)

- Width: 72
- Depth: 4096
- Always Enabled

■ Port details

- clka
- addra[11:0]: Write address
- dina[71:0]: Write input
- wea[0:0]: Write enable



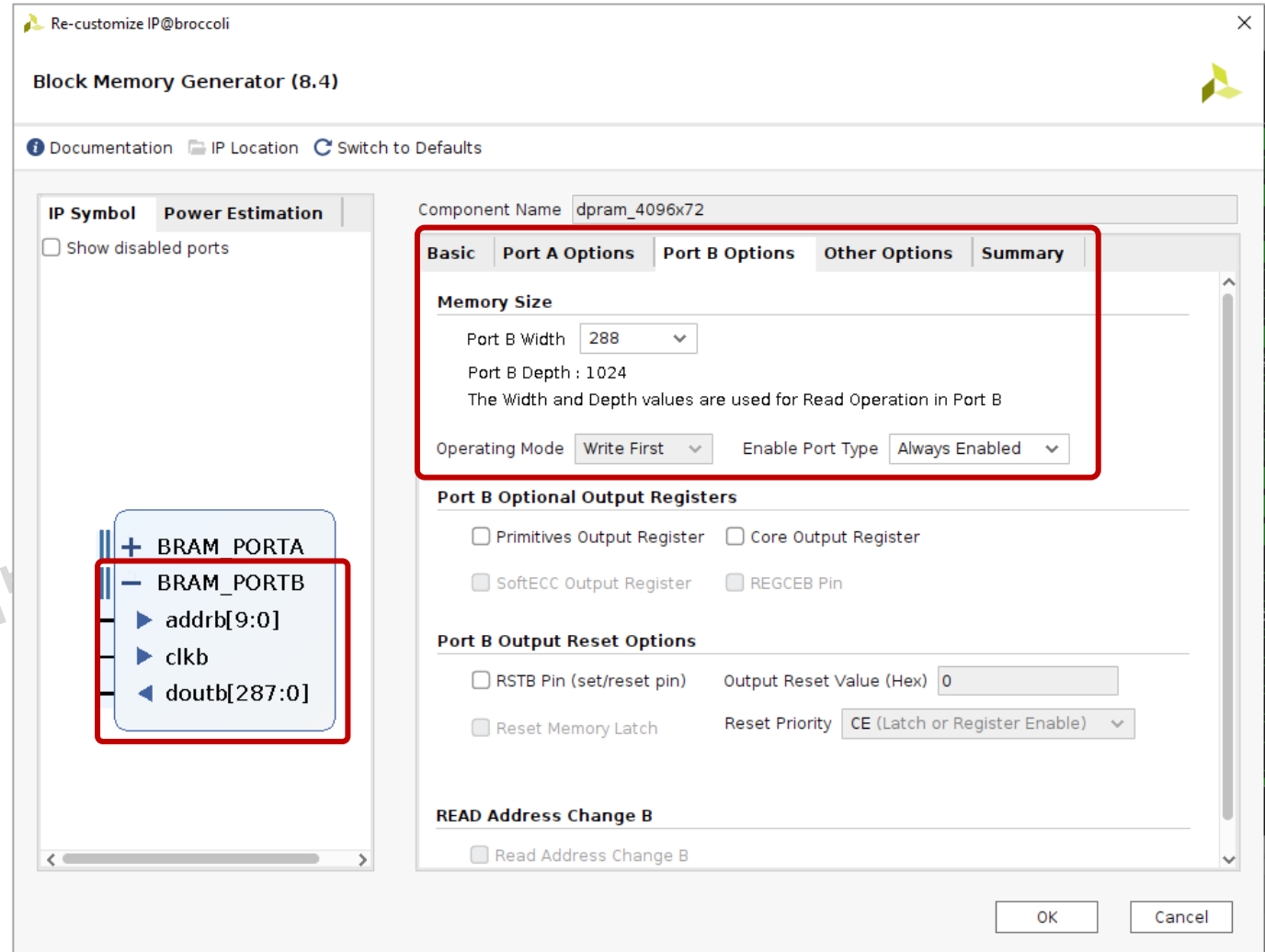
IP Configuration: Weight Buffer

■ Port B (Read Only)

- Width: 288
- Depth: 1024
- Always Enabled
- No primitives Output Reg.

■ Port details

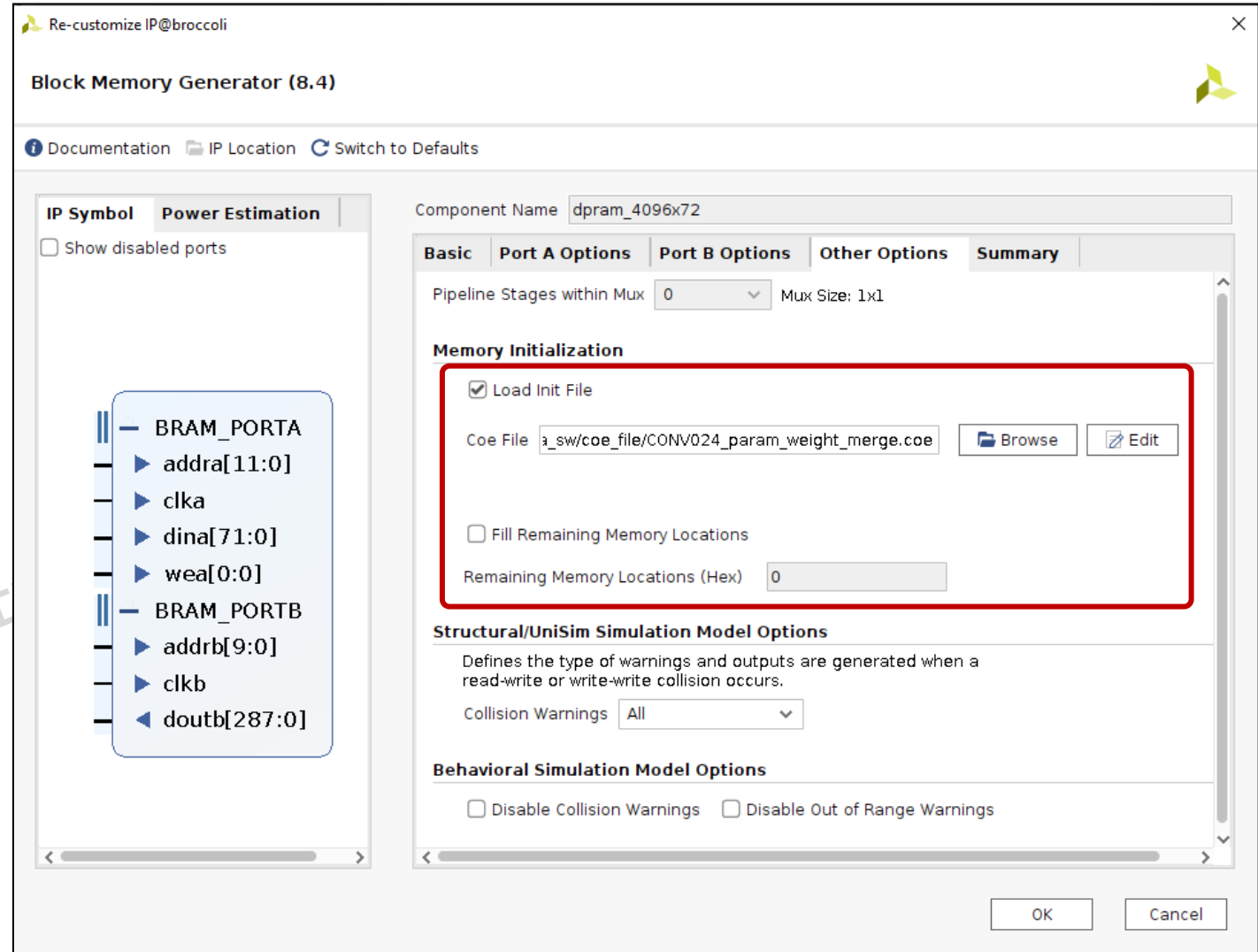
- clkb
- addrb[9:0]: Read address
- douta[287:0]: Read output



IP Configuration: Weight Buffer

■ Initialization

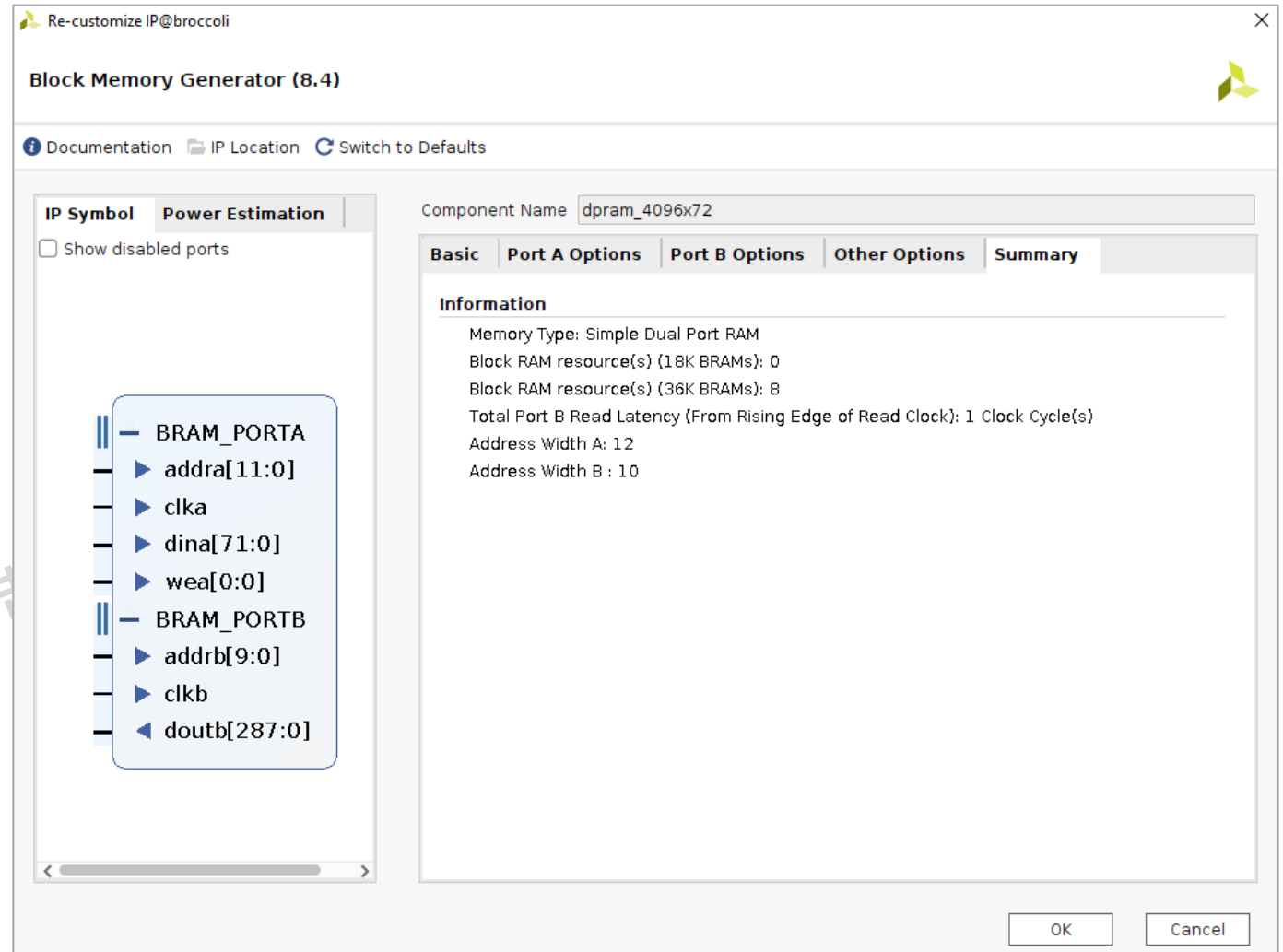
- Choose “Load Init File”
- Browse to the coe file
- Choose “Fill Remaining Memory Locations” with a default value.



IP Configuration: Weight Buffer

■ Summary

- Type: Simple Dual-Port
- Usage: Eight 36Kb BRAMs
- Read Latency: one cycle
- Address Width A: 12
- Address Width B: 10



Parameter Buffers (gbuff_param.v)

■ Inputs

- clk, rstn: Clock and reset signals
- dma_wea, dma_addra, dma_dina: Write request signals to update weights from DMA
- wgt_addrb, bsc_addra: Addresses from Controller to access weight, bias, and scale
- wgt_doutb: return weight; bsc_douta: return bias/scale

```
1  `timescale 1ns / 1ps
2  module gbuff_param(
3      input          clk,
4      // From DMA: Weight
5      input          dma_wea,
6      input [11:0]   dma_addra,
7      input [71:0]   dma_dina,
8
9      // From MAC: Weights
10     input [ 9:0]    wgt_addrb,
11     output [287:0] wgt_doutb,
12     // From MAC: Bias/Scale
13     input [11:0]    bsc_addra,
14     output [31:0]   bsc_douta
15 );
16
```

```
20  dpram_4096x72 u_buffer_weight(
21     /*input          */clka (clk      ),
22     /*input          */wea  (dma_wea  ),
23     /*input [ 11:0] */addra(dma_addra),
24     /*input [ 71:0] */dina  (dma_dina ),
25     /*input          */clkb (clk      ),
26     /*input [  9:0] */addrb(wgt_addrb),
27     /*input [287:0] */doutb(wgt_doutb)
28 );
29
30  spram_2560x32 u_buffer_bias_scale(
31     /*input          */clka (clk      ),
32     /*input          */wea  (1'b0     ),
33     /*input [11:0]   */addra(bsc_addra),
34     /*input [31:0]   */dina  (32'd0    ),
35     /*output [31:0]  */douta(bsc_douta)
36 );
```

Testbench (gbuff_param_tb.v)

- Design Under Test (DUT): gbuff_param
 - reg: clk, rstn, dma_***: from DMA, wgt_***: weights, bsc_**: Bias/Scale
 - wire: wgt_doutb: readout weight, bsc_douta: return bias/scale
- Clock: 100 MHz (period = 10 ns)

```
1  `timescale 1ns / 1ns
2
3  module gbuff_param_tb;
4  // Clock
5  parameter CLK_PERIOD = 10; //100MHz
6  reg clk;
7  reg rstn;
8  // From DMA: Weight
9  reg      dma_wea;
10 reg [11:0] dma_addra;
11 reg [71:0] dma_dina;
12 // From MAC: Weights
13 reg [ 9:0] wgt_addrb;
14 wire [287:0] wgt_doutb;
15 // From MAC: Bias/Scale
16 reg [11:0] bsc_addra;
17 wire [31:0] bsc_douta;
18
```

```
19 //-----
20 // DUT
21 //-----
22 gbuff_param u_gbuff_param(
23     /*input      */clk      (clk      ),
24     /*input      */dma_wea  (dma_wea  ),
25     /*input [11:0] */dma_addra(dma_addra),
26     /*input [31:0] */dma_dina (dma_dina ),
27     /*input [ 9:0] */wgt_addrb(wgt_addrb),
28     /*output [287:0] */wgt_doutb(wgt_doutb),
29     /*input [11:0] */bsc_addra(bsc_addra),
30     /*output [31:0] */bsc_douta(bsc_douta)
31 );
32
33 initial begin
34     clk = 1'b1;
35     forever #(CLK_PERIOD/2) clk = ~clk;
36 end
```

Testbench (gbuff_param_tb.v)

- Initialization: Set registers to zeros
- Send a trigger to a counter (trigger → run_counter = 1)
 - Running a counter (counter: 0, 1, 2, ..., 15)
 - ~address generator: (Address (wgt_addrb, bsc_addr): 0, 1, 2, ..., 15)
 - Return the data

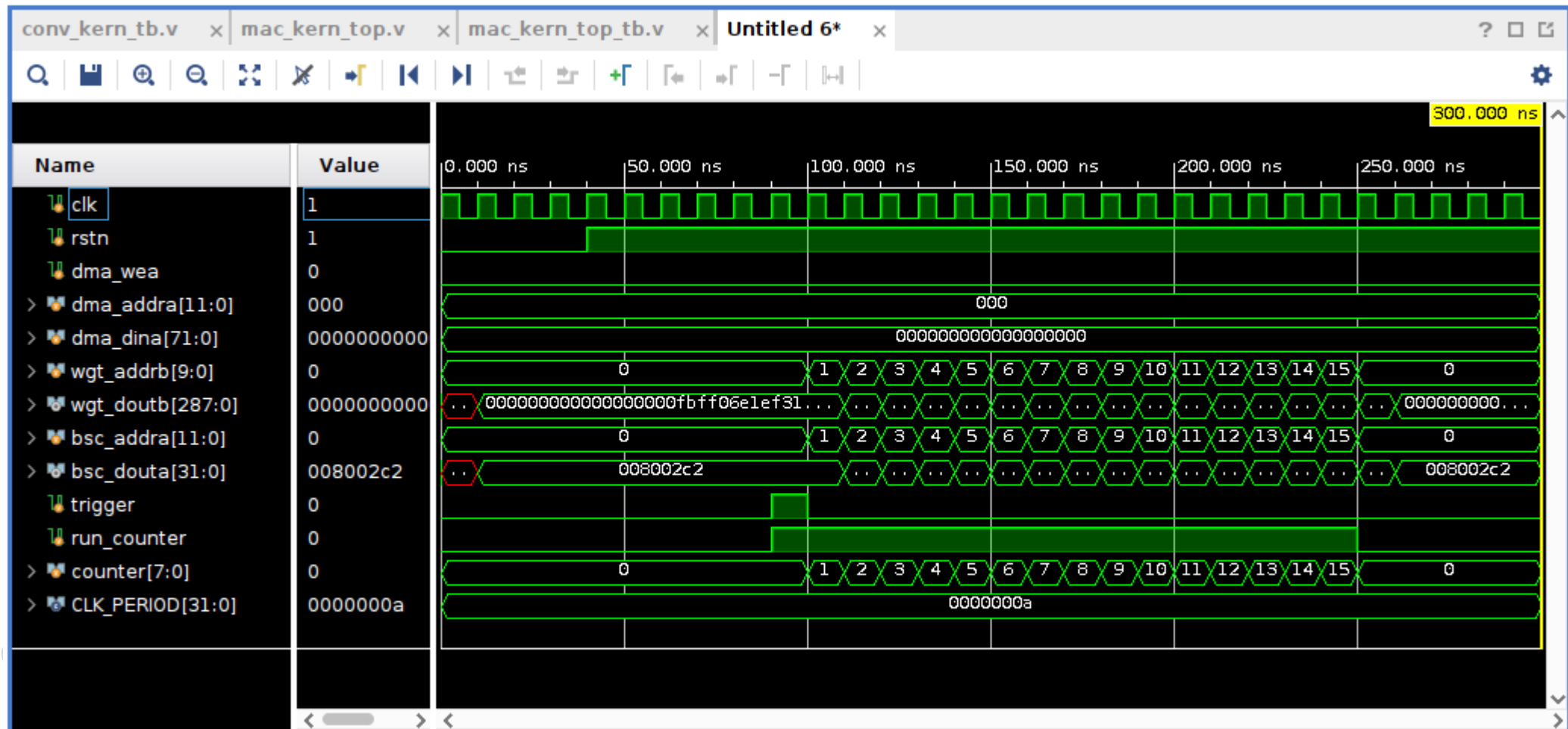
```
38 reg trigger;
39 reg run_counter;
40 reg [7:0] counter;
41
42 // Test cases
43 initial begin
44     rstn = 1'b0;    // Reset, low
45     dma_wea = 0;
46     dma_addra = 0;
47     dma_dina = 0;
48
49     trigger = 0;
50     #(4*CLK_PERIOD) rstn = 1'b1;
51
52     1  #(4*CLK_PERIOD) trigger = 0;
53     #( CLK_PERIOD) trigger = 1;
54     #( CLK_PERIOD) trigger = 0;
55
56 end
```

```
58 always@(*) begin
59     wgt_addrb = counter;
60     bsc_addra = counter; 4
61 end
62
63 always @(posedge clk or negedge rstn) begin
64     if(!rstn) begin
65         run_counter <= 0;
66     end
67     else begin
68         2 if(trigger && (counter == 0 || counter == 15))
69             run_counter <= 1;
70         else if(counter == 15)
71             run_counter <= 0;
72     end
73 end
74
```

```
75 always @(posedge clk or negedge rstn) begin
76     if(!rstn) begin
77         counter <= 0;
78     end
79     else begin
80         if(run_counter) begin
81             if(counter == 15)
82                 counter <= 0;
83             3 else
84                 counter <= counter + 1;
85         end
86     end
87 end
```

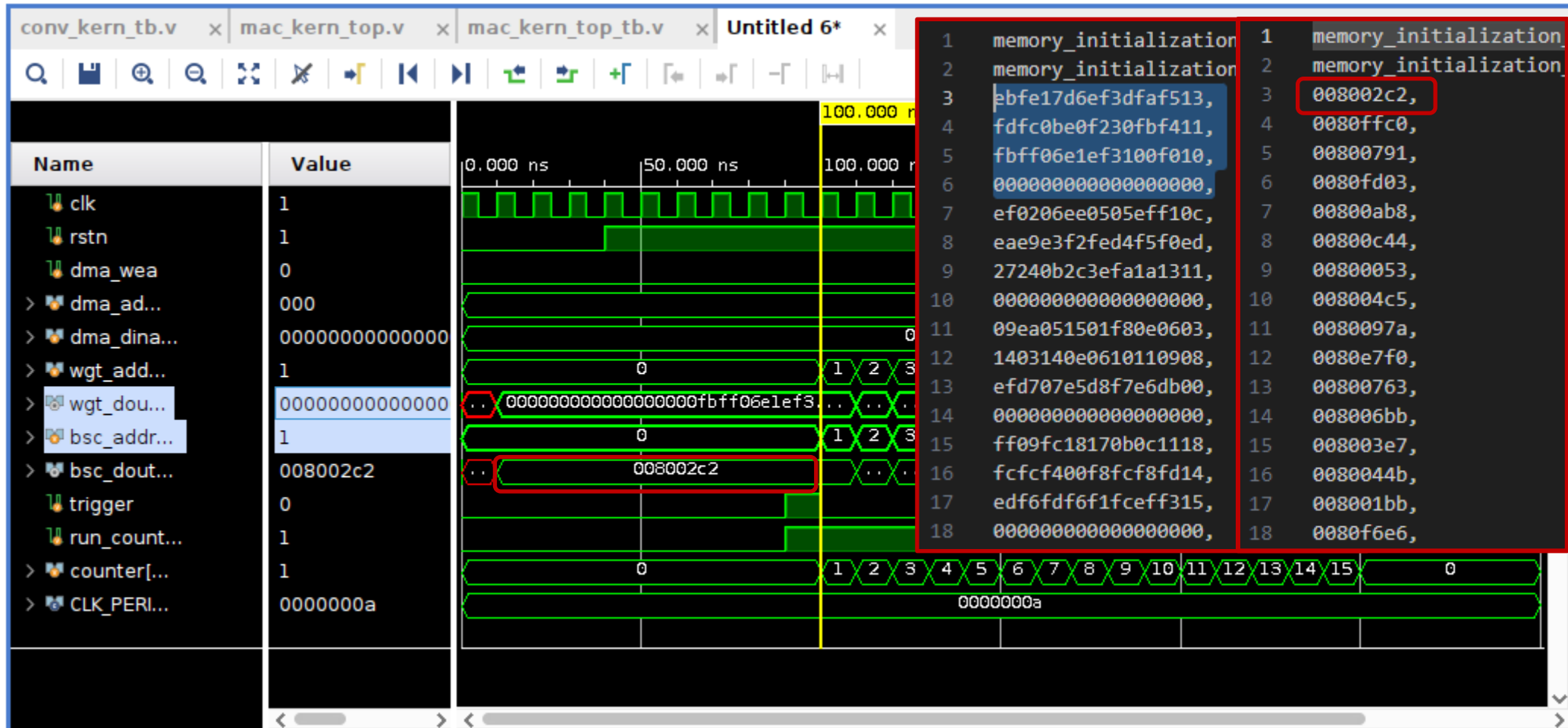

Simulation: Waveform (time = 300 ns)

- Counter values and addresses are: 0, 1, ..., 15



Simulation: Waveform (time = 300 ns)

- Return output, for example, 0080_02c2 @ addr=0



Summary

- After this lecture, you understand:
 - The top module design
 - Simulation with an external memory model
 - Design multipliers and MACs with DSPs from IP catalog
 - Design a buffer with block RAM from IP catalog
 - Conduct the functional and timing simulation
- Now you can reproduce the results.
- Based on it, you can make your own design

Q&A

Copyright 2022-2026. (차세대반도체 혁신공유대학 사업단) all rights reserved.